Day 5 — RAG + Vector Memory for Agents

> 🎯 **Goal**: Integrate long-term knowledge into the agent.

> ⏱️ **Time**: 1.5h theory + 3h practice

> 📦 **Deliverable**: Agent with working RAG + dynamic, long-term memory

---

Table of Contents

1. Why RAG?
2. Vector Retrieval Fundamentals
3. Chunking Strategies
4. Embedding Models
5. Hybrid Search
6. Context Window Management
7. Practice: Build a RAG Pipeline
8. Appendix: RAG Products Reference

Why RAG?

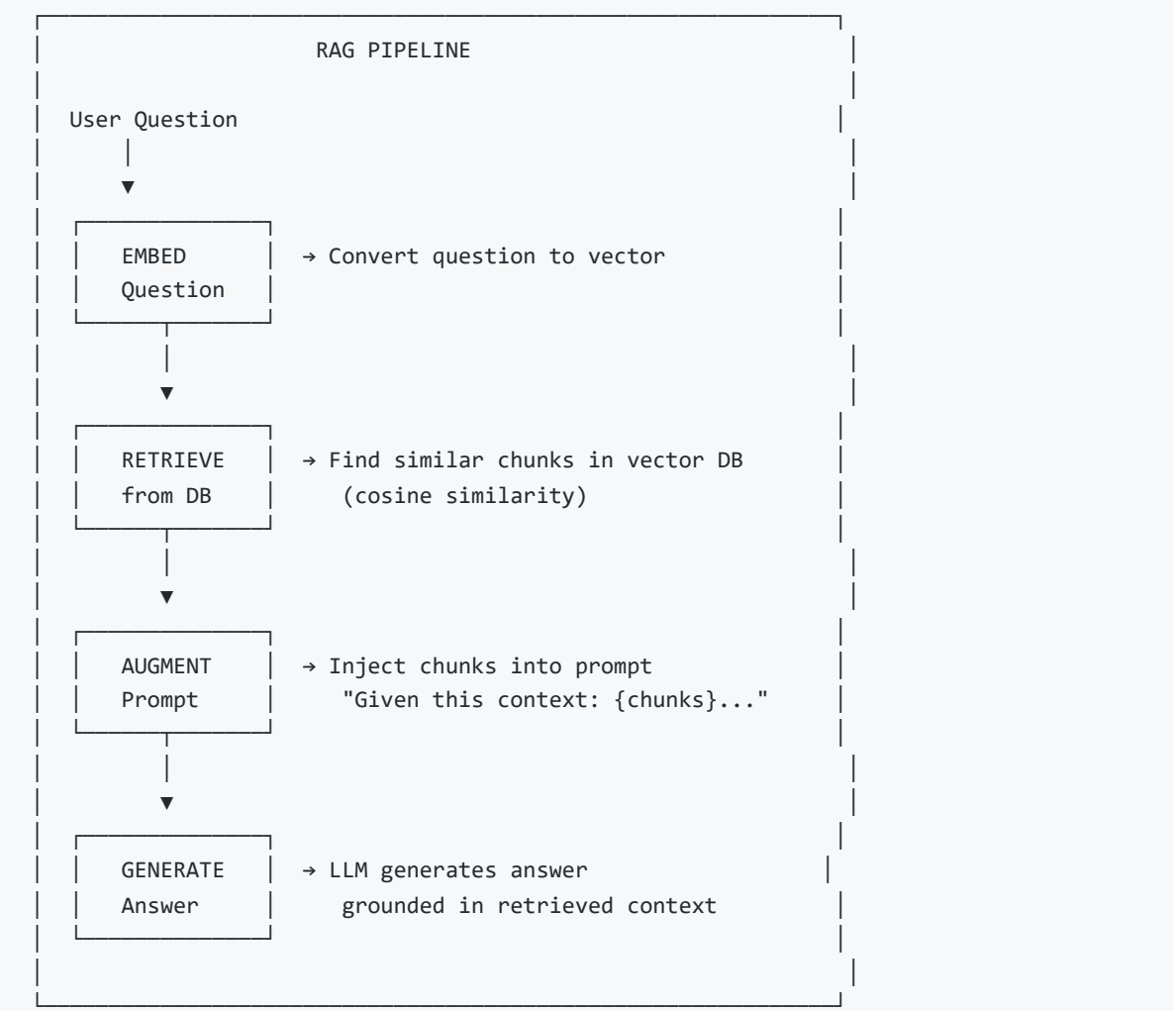
RAG = Retrieval-Augmented Generation. Instead of relying on the LLM's training data (which is frozen at a point in time), RAG lets the model **look up** information from external documents before answering.

```
Without RAG:
  Q: "What was our Q4 2024 revenue?"
  A: "I don't have access to your company's financial data."
      (or worse, hallucinated: "Your Q4 revenue was $12.5M")

With RAG:
  Q: "What was our Q4 2024 revenue?"

  1. RETRIEVE: Search company documents → find Q4 report
  2. AUGMENT:  Inject relevant chunks into the prompt
  3. GENERATE: LLM answers using the retrieved context

  A: "According to the Q4 2024 financial report, revenue was
      $18.3M, up 15% from Q3." ← Grounded in real data
```



RAG vs Fine-Tuning

	RAG	Fine-Tuning
Data freshness	Real-time updates	Frozen at training
Cost	Low (just embeddings)	High (GPU training)
Setup time	Hours	Days-weeks
Accuracy	High (grounded)	Variable
Hallucinations	Lower (has source)	Can still hallucinate
Data volume	Any amount	Needs curated dataset
Use case	Knowledge retrieval	Behavior change
Updatable	Add/remove docs live	Retrain required

When to use RAG:
- ✓ Company documents, knowledge bases
- ✓ Data that changes frequently
- ✓ When you need source citations
- ✓ Multi-domain knowledge

When to use Fine-Tuning:
- ✓ Specific output style/format

```
  ✓ Domain-specific reasoning (medical, legal)
  ✓ When RAG context window is insufficient
```

Vector Retrieval Fundamentals

What Are Embeddings?

Embeddings convert text into **dense numerical vectors** that capture semantic meaning. Similar texts → similar vectors.

```
Text: "The cat sat on the mat"     → [0.23, -0.15, 0.87, 0.42, ...]
Text: "A kitten rested on the rug" → [0.21, -0.13, 0.85, 0.39, ...]
                                      ↑ Very similar! (synonyms)

Text: "Stock prices rose sharply"   → [-0.45, 0.72, -0.11, 0.33, ...]
                                      ↑ Very different (different topic)

Key properties:
  - Dimension: typically 384–1536 floats
  - Semantic similarity → vector proximity
  - Language-agnostic (similar concepts in different languages map closely)
```

Cosine Similarity

The standard measure for comparing embeddings:

```
                    A · B          Σ(Ai × Bi)
cos(θ) = ──────────────────── = ──────────────────
           ||A|| × ||B||      √Σ(Ai²) × √Σ(Bi²)

Values:
   1.0 = identical meaning
   0.0 = completely unrelated
  -1.0 = opposite meaning (rare in practice)

Example:
  A = [0.23, -0.15, 0.87]   ("cat on mat")
  B = [0.21, -0.13, 0.85]   ("kitten on rug")

  A·B = (0.23×0.21) + (-0.15×-0.13) + (0.87×0.85) = 0.0483 + 0.0195 + 0.7395 = 0.8073
  ||A|| = √(0.0529 + 0.0225 + 0.7569) = √0.8323 = 0.9123
  ||B|| = √(0.0441 + 0.0169 + 0.7225) = √0.7835 = 0.8852

  cos(θ) = 0.8073 / (0.9123 × 0.8852) = 0.8073 / 0.8076 = 0.9996

  Very similar! ✓
```

Approximate Nearest Neighbor (ANN) Algorithms

Exact nearest-neighbor search over millions of vectors is too slow (O(n)) — you'd compare the query against **every** vector. ANN algorithms trade a tiny amount of accuracy (typically 95-99% recall) for massive speed gains (100-1000× faster).

```
The Core Problem:
  You have 10 million document vectors (each 1536 dimensions).
  A user query arrives → embed it → find the 5 closest vectors.

  Exact Search (brute force):
    Compare query with ALL 10M vectors → O(n) → ~seconds per query ❌

  ANN Search:
    Use a smart index to skip most vectors → O(log n) → ~milliseconds ✓
```

Algorithm 1: HNSW (Hierarchical Navigable Small World)

The most popular ANN algorithm. Used by **ChromaDB**, **Qdrant**, **pgvector**, **Weaviate**, **Pinecone**.

Core Idea: Build a multi-layer graph. Top layers are sparse (long-range connections for fast navigation). Bottom layers are dense (short-range connections for precision). Search starts at the top and "zooms in" layer by layer.

```
How HNSW Works — Think of it like a map with zoom levels:

Layer 2 (coarsest):    A ──────────────── D          ← few nodes, long jumps
                                                       (like a highway map)

Layer 1 (medium):      A ──── B ──── D ──── F        ← more nodes, shorter jumps
                                                       (like a regional map)

Layer 0 (finest):      A — B — C — D — E — F — G     ← all nodes, local connections
                                                       (like a street map)

Search for query Q (closest to E):
  1. Start at Layer 2 → enter at A → jump to D (closer to Q)
  2. Drop to Layer 1 → at D → jump to F (closer to Q)
  3. Drop to Layer 0 → at F → walk to E ✓ (nearest neighbor found!)

  Visited: A → D → F → E  (4 nodes out of 7 — skipped most!)
```

```
Key Parameters:
  M    = max connections per node (default: 16)
         Higher M → better recall, more memory, slower build
  efC  = construction beam width (default: 200)
         How many candidates to consider when building the graph
  efS  = search beam width (default: 50-200)
         Higher efS → better recall, slower search
```

```
          This is the main accuracy/speed knob at query time

Performance:
  Build time:  O(n · log n)
  Search time: O(log n)
  Memory:      O(n · M) — stores graph edges + original vectors
  Recall:      95-99% with proper tuning
```

```
When to use HNSW:
  ✅ Dataset fits in memory (< ~50M vectors)
  ✅ Need low-latency queries (< 10ms)
  ✅ Most RAG / semantic search use cases
  ❌ Dataset too large for RAM
  ❌ Need to update vectors very frequently (rebuild is expensive)
```

Algorithm 2: IVF (Inverted File Index)

Used by **FAISS** (Facebook AI Similarity Search). Great for very large datasets.

Core Idea: Pre-cluster all vectors into cells (using k-means). At query time, only search vectors in the nearest cells — skip everything else.

```
How IVF Works — Think of it like searching a library by section:

Step 1: BUILD — Cluster vectors into cells using k-means

  All 10M vectors → k-means with k=1000 → 1000 clusters (cells)

  Cell 1 (centroid C1):  [v1, v5, v23, v89, ...]     ~10K vectors
  Cell 2 (centroid C2):  [v2, v7, v41, v102, ...]     ~10K vectors
  ...
  Cell 1000 (centroid C1000): [v999, v1004, ...]      ~10K vectors

Step 2: SEARCH — Find nearest cells, then search within them

  Query Q arrives:
    1. Compare Q with 1000 centroids → find nearest nprobe cells
       (nprobe=10: search 10 cells out of 1000)
    2. Exhaustive search within those 10 cells (~100K vectors)
    3. Return top-k results

  Before: search 10,000,000 vectors
  After:  search     100,000 vectors  ← 100× faster!
```

```
Visual Example (2D, k=4 clusters, nprobe=1):

          ┌──────────────────┐
          │  C2              │
          │   . . .          │
```

```
    ┌────┤  · · ·           │
    │ C1 │──────────────────┤
    │ · ·│                  │
    │· · │  C4              │
    │ · ·│   · · ·     ★ Q  │  ← Query lands near C4
    ├────┤  · · · ·         │
    │ C3 │  · · ·           │
    │· · │                  │
    │ · ·│                  │
    └────┴──────────────────┘

    Step 1: Q is closest to centroid C4
    Step 2: Search only C4's vectors (skip C1, C2, C3)
    ⚠️ Risk: True nearest neighbor might be in C3 (near border)
         → Fix: increase nprobe to also search neighboring cells
```

```
Key Parameters:
  nlist  = number of clusters/cells (typical: √n to 4√n)
           1M vectors → nlist = 1000–4000
  nprobe = number of cells to search at query time
           Higher nprobe → better recall, slower search
           nprobe=1:   fast but may miss neighbors near borders
           nprobe=10:  good balance for most cases
           nprobe=100: high recall, approaching brute force

Performance:
  Build time:  O(n · k · iterations)  — k-means training
  Search time: O(nprobe · n/nlist)    — ~O(√n) with typical settings
  Memory:      O(n · d)  — stores all vectors + centroids
  Recall:      90-99% depending on nprobe
```

Algorithm 3: Product Quantization (PQ)

A **compression** technique — reduces memory by 4-64×. Often combined with IVF (→ IVF-PQ). Used by **FAISS**.

Core Idea: Split each high-dimensional vector into sub-vectors, then replace each sub-vector with the ID of its nearest centroid in a learned codebook. This compresses each vector from e.g. 6KB → 96 bytes.

```
How PQ Works — Think of it like lossy image compression for vectors:

Original vector (1536 dimensions, float32 → 6,144 bytes):
  [0.23, -0.15, 0.87, 0.42, -0.31, 0.55, ..., 0.12, -0.08]
   \_____________________________/  \_____________________________/
         1536 floats = 6,144 bytes per vector
         × 10M vectors = ~60 GB RAM 😱

Step 1: Split vector into m=8 sub-vectors (each 192 dims):
  [0.23, -0.15, ...] [0.87, 0.42, ...] ... [0.12, -0.08, ...]
   sub-vector 1       sub-vector 2          sub-vector 8
```

```
Step 2: For each sub-space, learn 256 centroids via k-means:
  Sub-space 1: 256 centroids → codebook 1
  Sub-space 2: 256 centroids → codebook 2
  ...
  Sub-space 8: 256 centroids → codebook 8

Step 3: Replace each sub-vector with its nearest centroid ID (1 byte):
  [0.23, -0.15, ...] → centroid #42  → stored as: 42
  [0.87, 0.42, ...] → centroid #187 → stored as: 187
  ...
  [0.12, -0.08, ...] → centroid #7   → stored as: 7

Compressed vector: [42, 187, 93, 5, 211, 156, 78, 7]  ← 8 bytes!

  Before: 6,144 bytes × 10M = 60 GB
  After:      8 bytes × 10M = 80 MB  ← 750× compression! 🎉
```

```
IVF-PQ (the winning combo for billion-scale search):
  ┌──────────────────────────────────────────────┐
  │ 1. IVF narrows search to a few clusters      │
  │ 2. PQ compresses vectors so they fit in memory  │
  │ 3. Search uses Asymmetric Distance Computation  │
  │    (exact query vs. compressed database vectors) │
  └──────────────────────────────────────────────┘

  Example: 1 billion vectors × 1536 dims
    Brute force:  6 TB RAM, seconds per query
    IVF-PQ:       ~8 GB RAM, milliseconds per query
```

```
Key Parameters:
  m         = number of sub-vectors (typical: 8, 16, 32)
              More sub-vectors → more bytes per code → better accuracy
  nbits     = bits per centroid (typical: 8 → 256 centroids per sub-space)
  code_size = m × (nbits / 8) bytes per vector

Performance:
  Build time:  O(n · m · 256 · iterations)  — train codebooks
  Search time: O(m · 256 + k)  — lookup table + scan
  Memory:      O(n · m) — massive savings!
  Recall:      85-95% (lower than HNSW/IVF, but uses far less memory)
```

Comparison: Which ANN Algorithm to Use?

Algorithm	Speed	Memory	Recall	Best For
HNSW	O(log n) ~1-5ms	High	95-99%	Default choice for most RAG apps (< 50M vectors)

```
| IVF           | O(√n)       | Medium    | 90-99%    | Large datasets where       |
|               | ~2-10ms     |           |           | build time is flexible     |
├───────────────┼─────────────┼───────────┼───────────┼────────────────────────────┤
| PQ            | Fast scan   | Very Low  | 85-95%    | Memory-constrained envs    |
|               |             | (8-64×)   |           | or billion-scale datasets  |
├───────────────┼─────────────┼───────────┼───────────┼────────────────────────────┤
| IVF-PQ        | O(√n)       | Very Low  | 88-97%    | Billion-scale search       |
|               | ~5-20ms     |           |           | (FAISS production default) |
├───────────────┼─────────────┼───────────┼───────────┼────────────────────────────┤
| Brute Force   | O(n)        | High      | 100%      | Small datasets (< 100K)    |
|               | ~100ms+     |           |           | or accuracy validation     |
└───────────────┴─────────────┴───────────┴───────────┴────────────────────────────┘

Decision Flowchart:

  How many vectors?
  │
  ├─ < 100K  → Brute force (exact) — fast enough, why approximate?
  │
  ├─ 100K - 50M → HNSW ← start here (most common)
  │
  ├─ 50M - 1B → IVF + HNSW or IVF-PQ
  │
  └─ > 1B → IVF-PQ (or ScaNN / DiskANN for specialized needs)
```

```
Real-World Mapping:

  ChromaDB   → HNSW (via hnswlib)          → great for prototyping
  Qdrant     → HNSW + quantization options  → production-ready, Rust-based
  Weaviate   → HNSW + PQ                    → multi-modal support
  Pinecone   → proprietary (HNSW-based)     → fully managed SaaS
  pgvector   → HNSW + IVFFlat               → stays in PostgreSQL
  FAISS      → IVF, PQ, IVF-PQ, HNSW        → research-grade, most flexible
  Milvus     → IVF, HNSW, DiskANN           → distributed, billion-scale
```

Chunking Strategies

Before embedding documents, you must split them into **chunks** — smaller pieces that fit the embedding model's context window and provide focused, retrievable units.

Why Chunking Matters

```
Problem: Your document is 50 pages long.
  - Can't embed the whole thing (embedding models have limits: 512-8K tokens)
  - Even if you could, one giant vector wouldn't be specific enough
  - Retrieving the whole doc wastes context window on irrelevant parts

Solution: Split into chunks, embed each separately.
```

```
  50-page doc → 200 chunks → 200 vectors
  Query matches the 3 most relevant chunks → focused, specific context
```

Strategy 1: Fixed-Size Chunking

```
Split text into chunks of N tokens/characters with overlap.

Parameters:
  chunk_size: 500 tokens
  overlap: 50 tokens (10% overlap between consecutive chunks)

Document: "AAAA BBBB CCCC DDDD EEEE FFFF GGGG HHHH IIII JJJJ"

Chunk 1: "AAAA BBBB CCCC DDDD EEEE"
Chunk 2: "DDDD EEEE FFFF GGGG HHHH"   ← overlaps with chunk 1
Chunk 3: "GGGG HHHH IIII JJJJ"         ← overlaps with chunk 2

Why overlap?
  Without overlap, a sentence split across two chunks is lost.
  "The deadline is December 31" → "The deadline is" | "December 31"
  Neither chunk has the full information!
  With overlap, the full sentence appears in at least one chunk.

Pros: Simple, consistent chunk sizes
Cons: May split mid-sentence, mid-paragraph, mid-thought
```

Strategy 2: Recursive Character Splitting

```
Split by natural boundaries, recursively trying smaller separators.

Separator hierarchy:
  1. "\n\n" (paragraph boundaries)
  2. "\n" (line boundaries)
  3. ". " (sentence boundaries)
  4. " " (word boundaries)
  5. "" (character-level, last resort)

Algorithm:
  1. Try splitting by "\n\n"
  2. If any chunk > max_size, split that chunk by "\n"
  3. If still > max_size, split by ". "
  4. Continue down the hierarchy

Result: Chunks respect natural text boundaries
  Paragraph-level chunks when possible
  Sentence-level when paragraphs are too long

Pros: Preserves semantic units
Cons: Variable chunk sizes
```

Strategy 3: Semantic Chunking

```
Split based on topic/meaning changes, not character boundaries.

Algorithm:
  1. Split into sentences
  2. Embed each sentence
  3. Compare adjacent sentence embeddings
  4. When similarity drops below threshold → new chunk

Sentences:  S1    S2    S3    S4    S5    S6    S7    S8
Similarity:    0.9   0.85  0.82  0.3   0.88  0.91  0.4
                                 ↑                  ↑
                              Topic change!       Topic change!

Chunks: [S1,S2,S3,S4] | [S5,S6,S7] | [S8]

Pros: Best semantic coherence
Cons: Requires embedding each sentence (slow, expensive)
      Variable chunk sizes
```

Chunking Comparison

```
┌──────────────┬──────────────┬──────────────┬──────────────┬──────────────┐
│ Strategy     │ Quality      │ Speed        │ Consistency  │ Best For     │
├──────────────┼──────────────┼──────────────┼──────────────┼──────────────┤
│ Fixed-size   │ Medium       │ Very fast    │ Uniform size │ Large vol    │
│ Recursive    │ Good         │ Fast         │ Variable     │ General      │
│ Semantic     │ Best         │ Slow         │ Variable     │ Quality      │
└──────────────┴──────────────┴──────────────┴──────────────┴──────────────┘

Recommended defaults:
  chunk_size: 500 tokens (~2000 characters)
  overlap: 50-100 tokens (10-20%)
  strategy: Recursive (best balance)
```

Embedding Models

Popular Embedding Models

```
┌──────────────────────────┬──────┬──────┬──────────────────────────┐
│ Model                    │ Dims │ Max  │ Notes                    │
│                          │      │ Tok  │                          │
├──────────────────────────┼──────┼──────┼──────────────────────────┤
│ text-embedding-3-small   │ 1536 │ 8191 │ OpenAI, cheap, fast      │
│ text-embedding-3-large   │ 3072 │ 8191 │ OpenAI, best quality     │
│ text-embedding-ada-002   │ 1536 │ 8191 │ OpenAI, legacy           │
```

```
| all-MiniLM-L6-v2          | 384  | 256  | Free, local, fast      |
| all-mpnet-base-v2         | 768  | 512  | Free, local, better    |
| bge-large-en-v1.5         | 1024 | 512  | Free, local, top tier  |
| nomic-embed-text-v1.5     | 768  | 8192 | Free, long context     |
| Cohere embed-english-v3   | 1024 | 512  | API, good quality      |
└──────────────────────────┴──────┴──────┴────────────────────────┘

Choosing a model:
  - Need free/local?  → all-MiniLM-L6-v2 (fastest) or bge-large (best)
  - Need API?         → text-embedding-3-small (best value)
  - Need max quality? → text-embedding-3-large
  - Need long chunks? → nomic-embed-text-v1.5
```

Embedding Code Example

```python
# Option A: OpenAI embeddings
import openai

def embed_openai(texts: list[str], model="text-embedding-3-small") -> list:
    """Embed texts using OpenAI API."""
    response = openai.embeddings.create(
        model=model,
        input=texts
    )
    return [item.embedding for item in response.data]

# Option B: Local embeddings (free, no API key)
# pip install sentence-transformers
from sentence_transformers import SentenceTransformer

model = SentenceTransformer('all-MiniLM-L6-v2')

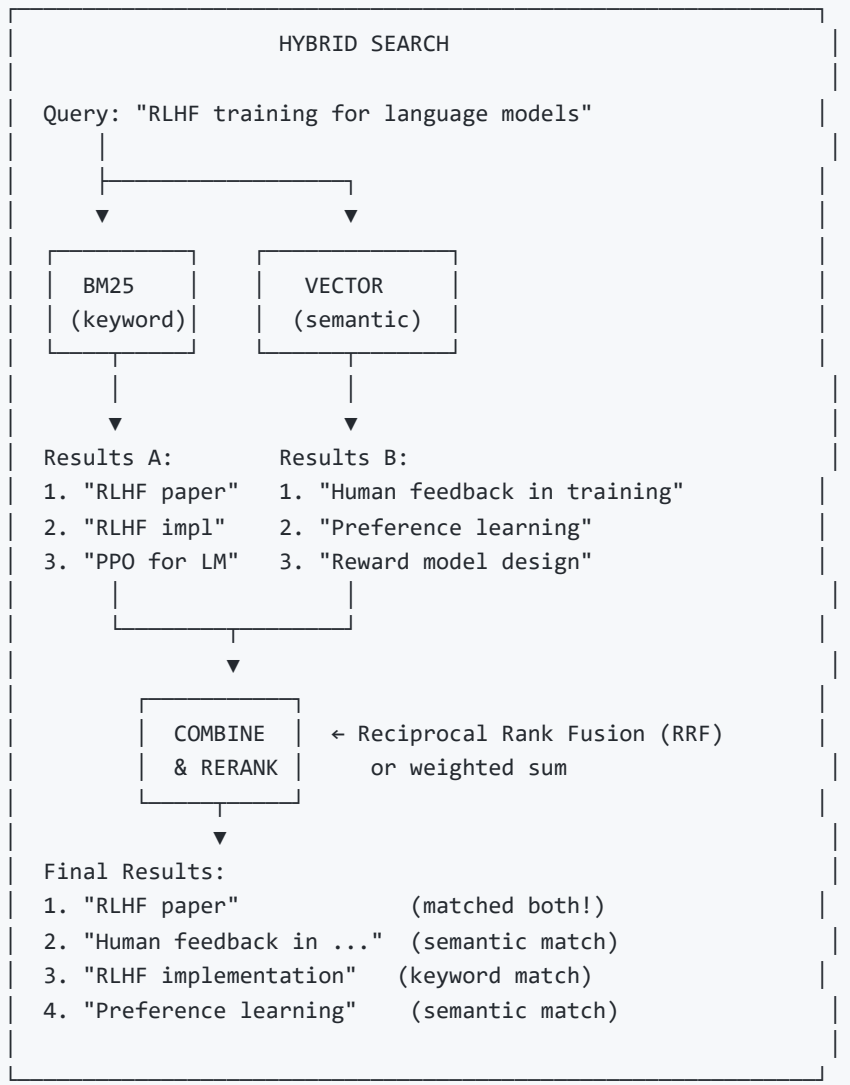
def embed_local(texts: list[str]) -> list:
    """Embed texts using local model."""
    embeddings = model.encode(texts, normalize_embeddings=True)
    return embeddings.tolist()

# Compare:
text = "How does vector search work?"
# OpenAI: 1536-dim vector, ~$0.00002 per text
# Local:  384-dim vector, free, ~10ms per text
```

Hybrid Search

Pure vector search misses exact keyword matches. Pure keyword search misses semantic similarity. **Hybrid search** combines both.

BM25 + Vector Search



BM25 Explained

```
BM25 is the classic information retrieval algorithm (used by Elasticsearch,
Lucene, etc.). It scores documents based on term frequency and rarity.

Score(D, Q) = Σ IDF(qi) × (f(qi, D) × (k1 + 1)) / (f(qi, D) + k1 × (1 - b + b × |D|/avgdl

Where:
  IDF(qi) = log((N - n(qi) + 0.5) / (n(qi) + 0.5))  ← How rare is this word?
  f(qi, D) = frequency of word qi in document D
  |D| = document length
  avgdl = average document length
  k1 = 1.2 (tuning parameter, controls term frequency saturation)
  b = 0.75 (tuning parameter, controls length normalization)

In plain English:
  "Score higher if the document contains rare query words,
```

```
   but don't over-reward having the same word many times,
   and penalize very long documents slightly."

BM25 is great at:
  ✓ Exact keyword matches ("RLHF", "Kubernetes", error codes)
  ✓ Rare/specific terms (product names, acronyms)
  ✓ Fast and well-understood

BM25 is bad at:
  ✗ Synonyms ("car" won't match "automobile")
  ✗ Paraphrases ("how to cook" won't match "recipe preparation")
  ✗ Conceptual similarity
```

Reciprocal Rank Fusion (RRF)

```
Combines rankings from multiple search methods:

RRF_score(d) = Σ 1 / (k + rank_i(d))

Where k = 60 (constant, prevents outlier dominance)

Example:
  Document "RLHF paper":
    BM25 rank: 1,  Vector rank: 4
    RRF = 1/(60+1) + 1/(60+4) = 0.0164 + 0.0156 = 0.0320

  Document "Preference learning":
    BM25 rank: 8,  Vector rank: 2
    RRF = 1/(60+8) + 1/(60+2) = 0.0147 + 0.0161 = 0.0308

  "RLHF paper" scores higher because it ranks well in BOTH methods.
```

Reranking

Initial retrieval (vector search, BM25, or hybrid) is fast but **coarse** — it uses lightweight similarity scores. Reranking adds a second, more accurate pass over the top-N results using a powerful model that reads the **query + document together**.

```
Why Reranking Matters:

  Without Reranking:
    Query: "How does RLHF work?"
    Top-5 from vector search:
      1. "RLHF paper abstract"            ← relevant ✓
      2. "List of RL algorithms"           ← tangential ✗
      3. "Human feedback collection"       ← relevant ✓
      4. "PPO training for language models" ← relevant ✓
      5. "Reward model architectures"      ← relevant ✓

  With Reranking (retrieve 20, rerank, keep top 5):
```

```
    Retrieve top-20 → reranker scores each → keep best 5:
      1. "RLHF paper abstract"             ← was #1
      2. "PPO training for language models" ← was #4
      3. "Step-by-step RLHF tutorial"      ← was #12 (rescued! 🎯)
      4. "Human feedback collection"       ← was #3
      5. "Reward model architectures"      ← was #5

  The reranker found a highly relevant doc at position #12 that
  vector search alone would have missed!
```

Bi-Encoder vs. Cross-Encoder

```
This is the key distinction that makes reranking work:

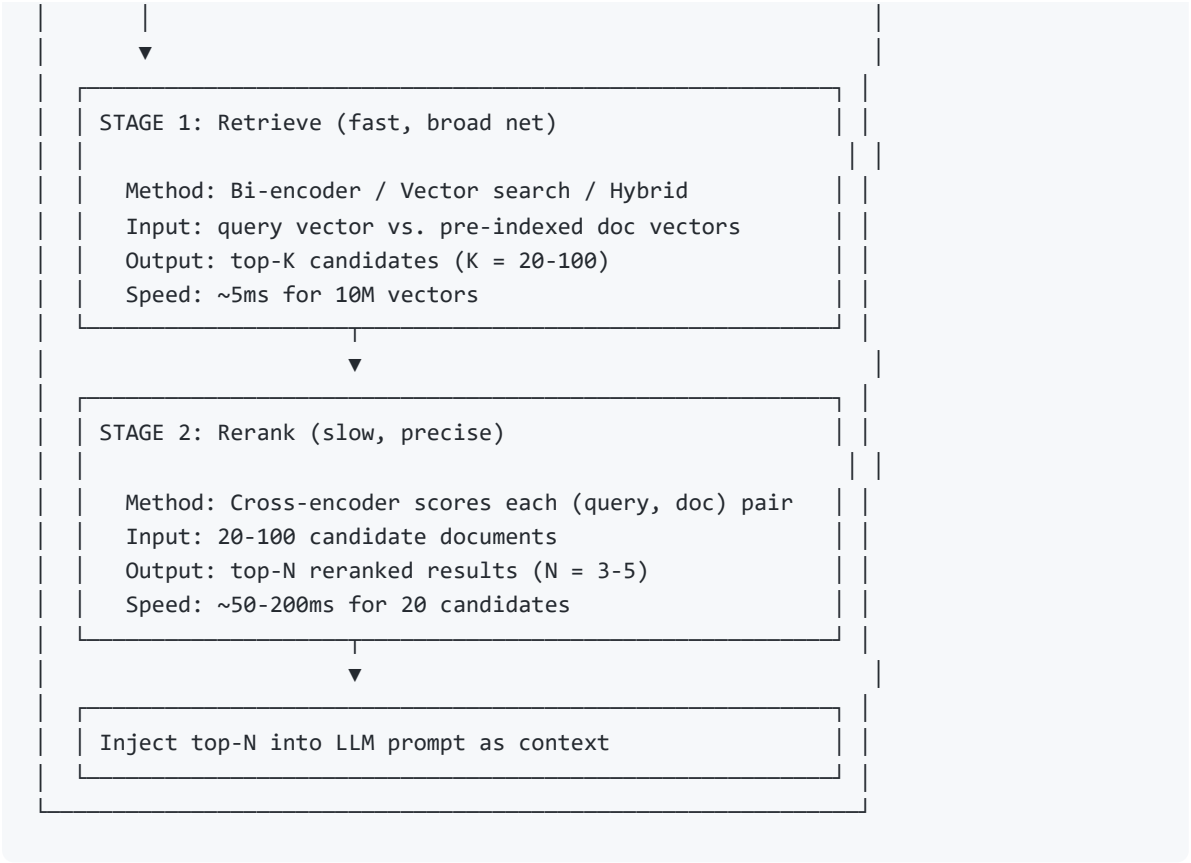
┌──────────────────────────────────────────────────────────────┐
│                    BI-ENCODER (Stage 1: Retrieval)           │
│                                                              │
│  Encodes query and documents INDEPENDENTLY                   │
│                                                              │
│  Query ──→ [Encoder] ──→ query_vector ──┐                    │
│                                         ├─→ cosine similarity │
│  Doc   ──→ [Encoder] ──→ doc_vector   ──┘                    │
│                                                              │
│  ✅ Fast: documents pre-encoded offline, only encode query once │
│  ✅ Scalable: works with millions of documents               │
│  ❌ No interaction between query and document during encoding │
│     → can miss nuanced relevance                             │
└──────────────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────────────┐
│                  CROSS-ENCODER (Stage 2: Reranking)          │
│                                                              │
│  Encodes query AND document TOGETHER (full attention)        │
│                                                              │
│  [Query + Doc] ──→ [Transformer] ──→ relevance_score (0 to 1)  │
│                                                              │
│  ✅ Much more accurate: sees both texts simultaneously       │
│  ✅ Captures word interactions, context, and nuance          │
│  ❌ Slow: must run for EACH query-document pair              │
│     → impractical for millions of docs, fine for top-20      │
└──────────────────────────────────────────────────────────────┘

Why not just use cross-encoder for everything?
  Cross-encoder on 10M docs = 10M forward passes per query → minutes ❌
  Bi-encoder on 10M docs + cross-encoder on top 20 = fast + accurate ✓
```

The Two-Stage Retrieval Pipeline

```
┌──────────────────────────────────────────────────────────┐
│                    TWO-STAGE RETRIEVAL                   │
│                                                          │
│  User Query: "How does RLHF improve LLM alignment?"      │
```



Reranking Models

Popular Reranking Models:

Model	Notes
Cohere Rerank	API-based. Easy to integrate. Best for quick production setup.
cross-encoder/ ms-marco-MiniLM-L-6-v2	Open-source (HuggingFace). Good balance of speed and accuracy.
BGE Reranker (bge-reranker-v2-m3)	Open-source (BAAI). State-of-the-art accuracy. Multilingual.
Jina Reranker	Open-source. Fast, good for long documents (8K token context).
LLM-as-reranker	Use GPT/Claude to score relevance. Most accurate but slowest + costly.

Code Example (using sentence-transformers):

```
from sentence_transformers import CrossEncoder

reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
```

```
    query = "How does RLHF work?"
    candidates = [
        "RLHF paper abstract...",
        "List of RL algorithms...",
        "Step-by-step RLHF tutorial...",
    ]

    # Score each (query, candidate) pair
    pairs = [(query, doc) for doc in candidates]
    scores = reranker.predict(pairs)
    # → [0.92, 0.15, 0.97]

    # Sort by score descending
    ranked = sorted(zip(scores, candidates), reverse=True)
    # → [(0.97, "Step-by-step RLHF tutorial..."),
    #    (0.92, "RLHF paper abstract..."),
    #    (0.15, "List of RL algorithms...")]
```

```
When to Use Reranking:

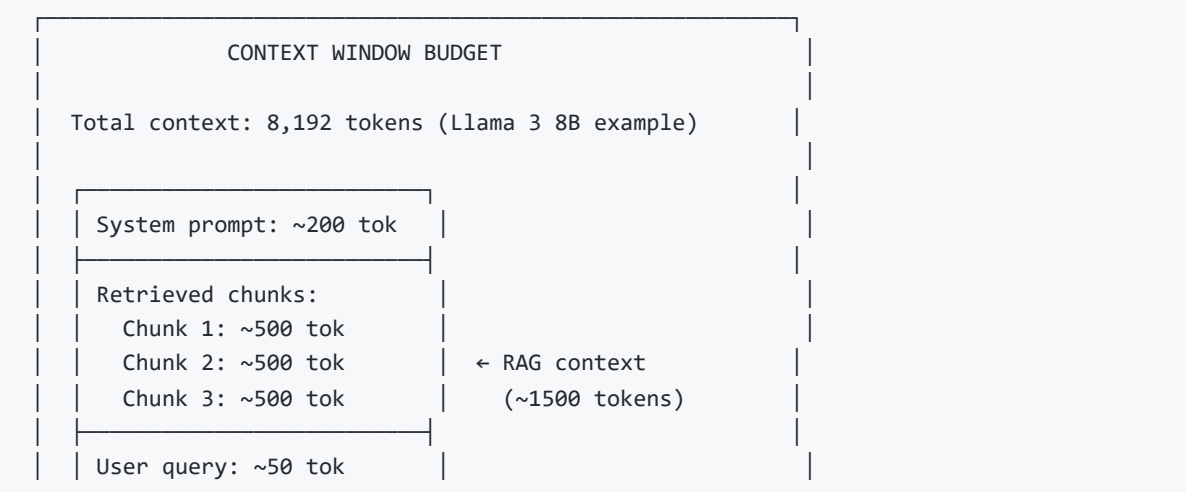
  ✅ Always — it almost always improves retrieval quality
  ✅ When initial retrieval returns noisy/tangential results
  ✅ When accuracy matters more than a few extra ms of latency
  ✅ Hybrid search + reranking is the gold standard for RAG

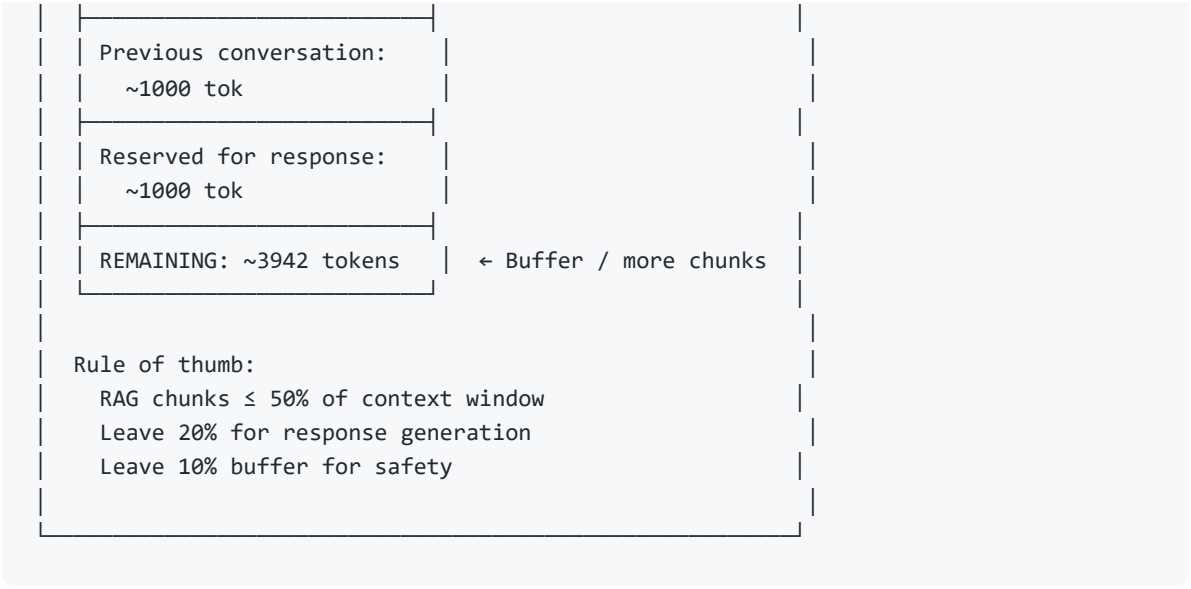
  ❌ Skip if latency budget is extremely tight (< 10ms total)
  ❌ Skip for trivial/small knowledge bases (< 100 documents)

  Typical improvement: +5-15% recall@5 over retrieval alone
```

Context Window Management

The retrieved chunks must fit in the LLM's context window along with the system prompt, user query, and room for the response.





Strategies for Managing Context

```
1. LIMIT CHUNKS: Retrieve only top-K (e.g., K=3-5)
   More chunks = more context but also more noise

2. RERANK: Retrieve top-20, rerank with a cross-encoder,
   keep top-5. Better quality than just top-5.

3. SUMMARIZE: If chunks are long, summarize them before
   injecting. Trades detail for fitting more information.

4. MAP-REDUCE: For very large retrievals:
   - Map: Ask LLM to extract relevant info from each chunk
   - Reduce: Combine extractions into final answer

5. SLIDING WINDOW: For conversation history:
   - Keep last N turns
   - Summarize older turns into a compact paragraph
```

Practice: Build a RAG Pipeline

Complete RAG Implementation

```
"""
RAG Pipeline — Complete implementation with chunking,
embedding, retrieval, and grounding evaluation.

Architecture:
  Documents → Chunker → Embedder → Vector Store
                                        |
```

```
  User Query → Embedder → Similarity Search → Top-K Chunks
                                                    |
                                              Prompt Builder → LLM → Answer
                                                    |
                                              Grounding Evaluator → Score
"""
import json
import math
import re
import hashlib
from dataclasses import dataclass, field
from typing import Optional


# ═══════════════════════════════════════════════════════════════
# SECTION 1: CHUNKING
# ═══════════════════════════════════════════════════════════════

@dataclass
class Chunk:
    """A chunk of text with metadata."""
    id: str
    text: str
    source: str  # Document name/path
    index: int   # Position in document
    metadata: dict = field(default_factory=dict)
    embedding: list = field(default_factory=list)


class RecursiveChunker:
    """
    Splits text into chunks using recursive character splitting.

    Tries to split at natural boundaries:
    1. Double newlines (paragraphs)
    2. Single newlines
    3. Sentences (periods)
    4. Words (spaces)
    """

    def __init__(self, chunk_size: int = 500, chunk_overlap: int = 50):
        self.chunk_size = chunk_size
        self.chunk_overlap = chunk_overlap
        self.separators = ["\n\n", "\n", ". ", " ", ""]

    def chunk_text(self, text: str, source: str = "unknown") -> list:
        """Split text into overlapping chunks."""
        raw_chunks = self._split_recursive(text, self.separators)

        # Merge small chunks and add overlap
        merged = self._merge_with_overlap(raw_chunks)

        chunks = []
        for i, chunk_text in enumerate(merged):
            chunk_id = hashlib.md5(
                f"{source}:{i}:{chunk_text[:50]}".encode()
```

```
            ).hexdigest()[:12]

            chunks.append(Chunk(
                id=chunk_id,
                text=chunk_text.strip(),
                source=source,
                index=i,
                metadata={
                    "char_count": len(chunk_text),
                    "word_count": len(chunk_text.split())
                }
            ))

        return chunks

    def _split_recursive(self, text: str, separators: list) -> list:
        """Recursively split text using separator hierarchy."""
        if not separators:
            return [text]

        sep = separators[0]
        remaining_seps = separators[1:]

        if sep == "":
            # Character-level split as last resort
            return [text[i:i+self.chunk_size]
                    for i in range(0, len(text), self.chunk_size)]

        parts = text.split(sep)

        result = []
        current = ""

        for part in parts:
            candidate = current + sep + part if current else part

            if len(candidate) <= self.chunk_size:
                current = candidate
            else:
                if current:
                    result.append(current)

                if len(part) > self.chunk_size:
                    # This part is too big — split with next separator
                    sub_parts = self._split_recursive(part, remaining_seps)
                    result.extend(sub_parts)
                    current = ""
                else:
                    current = part

        if current:
            result.append(current)

        return result

    def _merge_with_overlap(self, chunks: list) -> list:
```

```
        """Add overlap between consecutive chunks."""
        if len(chunks) <= 1:
            return chunks

        result = []
        for i, chunk in enumerate(chunks):
            if i > 0 and self.chunk_overlap > 0:
                # Get overlap from end of previous chunk
                prev = chunks[i-1]
                overlap_text = prev[-self.chunk_overlap:]
                # Find word boundary
                space_idx = overlap_text.find(' ')
                if space_idx > 0:
                    overlap_text = overlap_text[space_idx+1:]
                chunk = overlap_text + " " + chunk

            result.append(chunk)

        return result


# ═══════════════════════════════════════════════════════════════
# SECTION 2: VECTOR STORE (In-Memory)
# ═══════════════════════════════════════════════════════════════

class SimpleVectorStore:
    """
    A minimal in-memory vector store.

    Uses brute-force cosine similarity (fine for < 100K vectors).
    In production, use ChromaDB, Pinecone, FAISS, or pgvector.
    """

    def __init__(self):
        self.chunks: list[Chunk] = []

    def add(self, chunks: list):
        """Add chunks with embeddings to the store."""
        for chunk in chunks:
            if not chunk.embedding:
                raise ValueError(f"Chunk {chunk.id} has no embedding")
            self.chunks.append(chunk)

    def search(self, query_embedding: list, top_k: int = 5) -> list:
        """Find the top-K most similar chunks."""
        if not self.chunks:
            return []

        scored = []
        for chunk in self.chunks:
            score = self._cosine_similarity(query_embedding, chunk.embedding)
            scored.append((chunk, score))

        scored.sort(key=lambda x: x[1], reverse=True)
        return scored[:top_k]
```

```python
    def _cosine_similarity(self, a: list, b: list) -> float:
        """Compute cosine similarity between two vectors."""
        dot_product = sum(x * y for x, y in zip(a, b))
        norm_a = math.sqrt(sum(x * x for x in a))
        norm_b = math.sqrt(sum(x * x for x in b))

        if norm_a == 0 or norm_b == 0:
            return 0.0

        return dot_product / (norm_a * norm_b)

    def __len__(self):
        return len(self.chunks)


# ============================================================
# SECTION 3: BM25 (KEYWORD SEARCH)
# ============================================================

class SimpleBM25:
    """
    Simple BM25 implementation for keyword-based retrieval.
    """

    def __init__(self, k1: float = 1.2, b: float = 0.75):
        self.k1 = k1
        self.b = b
        self.docs = []       # List of tokenized documents
        self.doc_chunks = [] # Corresponding chunks
        self.avgdl = 0       # Average document length
        self.idf = {}        # Inverse document frequency
        self.doc_count = 0

    def add(self, chunks: list):
        """Index chunks for BM25 search."""
        for chunk in chunks:
            tokens = self._tokenize(chunk.text)
            self.docs.append(tokens)
            self.doc_chunks.append(chunk)

        self.doc_count = len(self.docs)
        self.avgdl = sum(len(d) for d in self.docs) / max(self.doc_count, 1)
        self._compute_idf()

    def search(self, query: str, top_k: int = 5) -> list:
        """Search for documents matching the query."""
        query_tokens = self._tokenize(query)

        scores = []
        for i, doc_tokens in enumerate(self.docs):
            score = self._score(query_tokens, doc_tokens)
            scores.append((self.doc_chunks[i], score))

        scores.sort(key=lambda x: x[1], reverse=True)
        return scores[:top_k]
```

```
    def _tokenize(self, text: str) -> list:
        """Simple tokenization."""
        return re.findall(r'\w+', text.lower())

    def _compute_idf(self):
        """Compute IDF for all terms."""
        df = {}  # Document frequency
        for doc in self.docs:
            unique_terms = set(doc)
            for term in unique_terms:
                df[term] = df.get(term, 0) + 1

        for term, freq in df.items():
            self.idf[term] = math.log(
                (self.doc_count - freq + 0.5) / (freq + 0.5) + 1
            )

    def _score(self, query_tokens: list, doc_tokens: list) -> float:
        """Compute BM25 score for a document."""
        score = 0.0
        doc_len = len(doc_tokens)

        # Count term frequencies in document
        tf = {}
        for token in doc_tokens:
            tf[token] = tf.get(token, 0) + 1

        for term in query_tokens:
            if term not in self.idf:
                continue

            term_freq = tf.get(term, 0)
            idf = self.idf[term]

            numerator = term_freq * (self.k1 + 1)
            denominator = term_freq + self.k1 * (
                1 - self.b + self.b * doc_len / max(self.avgdl, 1)
            )

            score += idf * numerator / denominator

        return score


# ═══════════════════════════════════════════════════════════════
# SECTION 4: EMBEDDING (Local, No API Needed)
# ═══════════════════════════════════════════════════════════════

class SimpleEmbedder:
    """
    A simple TF-IDF based embedder for practice.
    No API key needed. In production, use OpenAI or sentence-transformers.

    This creates sparse embeddings using word frequencies — not as good
    as neural embeddings, but demonstrates the concept perfectly.
    """
```

```
    def __init__(self, dim: int = 256):
        self.dim = dim
        self.vocabulary = {}
        self.idf = {}
        self.fitted = False

    def fit(self, texts: list):
        """Build vocabulary from a corpus of texts."""
        # Count document frequency for each word
        df = {}
        n_docs = len(texts)

        all_words = set()
        for text in texts:
            words = set(re.findall(r'\w+', text.lower()))
            all_words.update(words)
            for word in words:
                df[word] = df.get(word, 0) + 1

        # Keep top N words by document frequency
        sorted_words = sorted(df.items(), key=lambda x: x[1], reverse=True)
        for i, (word, freq) in enumerate(sorted_words[:self.dim]):
            self.vocabulary[word] = i
            self.idf[word] = math.log(n_docs / (freq + 1)) + 1

        self.fitted = True

    def embed(self, text: str) -> list:
        """Create an embedding vector for text."""
        if not self.fitted:
            raise ValueError("Embedder not fitted. Call fit() first.")

        words = re.findall(r'\w+', text.lower())
        word_counts = {}
        for w in words:
            word_counts[w] = word_counts.get(w, 0) + 1

        vector = [0.0] * self.dim
        for word, count in word_counts.items():
            if word in self.vocabulary:
                idx = self.vocabulary[word]
                tf = count / max(len(words), 1)
                vector[idx] = tf * self.idf.get(word, 1.0)

        # L2 normalize
        norm = math.sqrt(sum(x*x for x in vector))
        if norm > 0:
            vector = [x / norm for x in vector]

        return vector

    def embed_batch(self, texts: list) -> list:
        """Embed multiple texts."""
        return [self.embed(text) for text in texts]
```

```python
# ═══════════════════════════════════════════════════════════════
# SECTION 5: RAG PIPELINE
# ═══════════════════════════════════════════════════════════════

class RAGPipeline:
    """
    Complete RAG pipeline with hybrid search and grounding evaluation.

    ┌─────────────────────────────────────────────────────────┐
    │                    RAG Pipeline                         │
    │                                                         │
    │  Ingestion:                                             │
    │    Documents → Chunker → Embedder → Vector Store        │
    │                                   → BM25 Index          │
    │                                                         │
    │  Query:                                                 │
    │    Question → Embedder → Vector Search ─┐               │
    │            → BM25 Search ──────────────┤               │
    │                                        ▼               │
    │                                  Hybrid Merge           │
    │                                       │                 │
    │                                  Build Prompt           │
    │                                       │                 │
    │                                  Generate Answer        │
    │                                       │                 │
    │                                  Evaluate Grounding     │
    │                                       │                 │
    │                                  Return Result          │
    │                                                         │
    └─────────────────────────────────────────────────────────┘
    """

    def __init__(self, chunk_size: int = 500, chunk_overlap: int = 50,
                 top_k: int = 5, hybrid_alpha: float = 0.5):
        """
        Args:
            chunk_size: Maximum chunk size in characters
            chunk_overlap: Overlap between chunks
            top_k: Number of chunks to retrieve
            hybrid_alpha: Balance between vector (1.0) and BM25 (0.0)
        """
        self.chunker = RecursiveChunker(chunk_size, chunk_overlap)
        self.vector_store = SimpleVectorStore()
        self.bm25 = SimpleBM25()
        self.embedder = SimpleEmbedder()
        self.top_k = top_k
        self.hybrid_alpha = hybrid_alpha
        self.all_chunks = []

    def ingest(self, documents: dict):
        """
        Ingest documents into the RAG pipeline.

        Args:
            documents: dict of {name: text} pairs
```

```
        """
        print(f"📥 Ingesting {len(documents)} documents...")

        # Chunk all documents
        all_chunks = []
        for name, text in documents.items():
            chunks = self.chunker.chunk_text(text, source=name)
            all_chunks.extend(chunks)
            print(f"   📄 {name}: {len(chunks)} chunks")

        self.all_chunks = all_chunks

        # Fit embedder on all chunk texts
        all_texts = [c.text for c in all_chunks]
        self.embedder.fit(all_texts)

        # Embed all chunks
        for chunk in all_chunks:
            chunk.embedding = self.embedder.embed(chunk.text)

        # Add to stores
        self.vector_store.add(all_chunks)
        self.bm25.add(all_chunks)

        print(f"✅ Ingested {len(all_chunks)} total chunks")
        print(f"   Vector store: {len(self.vector_store)} vectors")

    def query(self, question: str, verbose: bool = True) -> dict:
        """
        Query the RAG pipeline.

        Returns:
            dict with: answer, chunks, scores, grounding_score
        """
        if verbose:
            print(f"\n🔍 Query: {question}")

        # Step 1: Retrieve relevant chunks
        retrieved = self._hybrid_search(question)

        if verbose:
            print(f"\n📚 Retrieved {len(retrieved)} chunks:")
            for chunk, score in retrieved:
                preview = chunk.text[:80].replace('\n', ' ')
                print(f"   [{score:.3f}] ({chunk.source}) {preview}...")

        # Step 2: Build prompt with context
        context = "\n\n---\n\n".join(
            f"[Source: {chunk.source}]\n{chunk.text}"
            for chunk, _ in retrieved
        )

        prompt = self._build_prompt(question, context)

        # Step 3: Generate answer (simulated without API)
        answer = self._generate_answer(question, retrieved)
```

```
        # Step 4: Evaluate grounding
        grounding_score = self._evaluate_grounding(answer, retrieved)

        if verbose:
            print(f"\n💬 Answer: {answer}")
            print(f"\n📊 Grounding score: {grounding_score:.2f}")

        return {
            "question": question,
            "answer": answer,
            "chunks": [(c.text, c.source, s) for c, s in retrieved],
            "grounding_score": grounding_score,
            "prompt": prompt
        }

    def _hybrid_search(self, query: str) -> list:
        """Combine vector and BM25 search results."""
        # Vector search
        query_embedding = self.embedder.embed(query)
        vector_results = self.vector_store.search(query_embedding, self.top_k * 2)

        # BM25 search
        bm25_results = self.bm25.search(query, self.top_k * 2)

        # Reciprocal Rank Fusion
        rrf_scores = {}
        k = 60  # RRF constant

        for rank, (chunk, _) in enumerate(vector_results):
            rrf_scores[chunk.id] = rrf_scores.get(chunk.id, 0) + \
                self.hybrid_alpha / (k + rank + 1)

        for rank, (chunk, _) in enumerate(bm25_results):
            rrf_scores[chunk.id] = rrf_scores.get(chunk.id, 0) + \
                (1 - self.hybrid_alpha) / (k + rank + 1)

        # Build result list
        chunk_map = {c.id: c for c in self.all_chunks}
        results = [
            (chunk_map[cid], score)
            for cid, score in rrf_scores.items()
            if cid in chunk_map
        ]
        results.sort(key=lambda x: x[1], reverse=True)

        return results[:self.top_k]

    def _build_prompt(self, question: str, context: str) -> str:
        """Build the RAG prompt."""
        return f"""Answer the question based ONLY on the provided context.
If the context doesn't contain enough information, say "I don't have enough
information to answer this question."

Always cite which source document your answer comes from.
```

```
Context:
{context}

Question: {question}

Answer:"""

    def _generate_answer(self, question: str,
                         retrieved: list) -> str:
        """
        Generate an answer from retrieved chunks.

        In production, this would call an LLM. For practice,
        we use extractive summarization (pulling relevant sentences).
        """
        if not retrieved:
            return "I don't have enough information to answer this question."

        # Simple extractive approach: find sentences most relevant to query
        query_words = set(re.findall(r'\w+', question.lower()))

        scored_sentences = []
        for chunk, chunk_score in retrieved:
            sentences = re.split(r'[.!?]+', chunk.text)
            for sentence in sentences:
                sentence = sentence.strip()
                if len(sentence) < 20:
                    continue

                words = set(re.findall(r'\w+', sentence.lower()))
                overlap = len(query_words & words)
                score = overlap * chunk_score
                scored_sentences.append((sentence, score, chunk.source))

        scored_sentences.sort(key=lambda x: x[1], reverse=True)

        # Take top sentences
        top_sentences = scored_sentences[:3]

        if not top_sentences:
            return "I don't have enough information to answer this question."

        answer_parts = []
        sources_used = set()
        for sent, _, source in top_sentences:
            answer_parts.append(sent)
            sources_used.add(source)

        answer = ". ".join(answer_parts)
        sources = ", ".join(sources_used)
        return f"{answer}. (Sources: {sources})"

    def _evaluate_grounding(self, answer: str,
                            retrieved: list) -> float:
        """
        Evaluate how well the answer is grounded in retrieved chunks.
```

```
        Returns a score from 0.0 (not grounded) to 1.0 (fully grounded).

        Method: Check what fraction of answer words appear in the
        retrieved context.
        """
        if not answer or not retrieved:
            return 0.0

        # Get all words from answer (excluding common stop words)
        stop_words = {"the", "a", "an", "is", "are", "was", "were", "be",
                      "been", "being", "have", "has", "had", "do", "does",
                      "did", "will", "would", "could", "should", "may",
                      "might", "can", "shall", "to", "of", "in", "for",
                      "on", "with", "at", "by", "from", "as", "into",
                      "through", "during", "before", "after", "and", "but",
                      "or", "not", "no", "this", "that", "these", "those",
                      "it", "its", "i", "don", "t", "s", "sources"}

        answer_words = set(re.findall(r'\w+', answer.lower())) - stop_words

        # Get all words from retrieved chunks
        context_words = set()
        for chunk, _ in retrieved:
            context_words.update(
                set(re.findall(r'\w+', chunk.text.lower())) - stop_words
            )

        if not answer_words:
            return 1.0

        # What fraction of answer words appear in context?
        grounded_words = answer_words & context_words
        score = len(grounded_words) / len(answer_words)

        return round(score, 3)

    def add_memory(self, key: str, text: str):
        """
        Add a piece of information to the agent's long-term memory.
        This is how agents can store things they learn for later retrieval.
        """
        chunks = self.chunker.chunk_text(text, source=f"memory:{key}")

        # Refit embedder with new vocabulary
        all_texts = [c.text for c in self.all_chunks] + [c.text for c in chunks]
        self.embedder.fit(all_texts)

        # Re-embed everything (in production, you'd use an incremental approach)
        for chunk in self.all_chunks:
            chunk.embedding = self.embedder.embed(chunk.text)
        for chunk in chunks:
            chunk.embedding = self.embedder.embed(chunk.text)

        self.all_chunks.extend(chunks)
```

```
        # Rebuild stores
        self.vector_store = SimpleVectorStore()
        self.vector_store.add(self.all_chunks)
        self.bm25 = SimpleBM25()
        self.bm25.add(self.all_chunks)

        print(f"🧠 Added memory '{key}': {len(chunks)} chunks")


# ═══════════════════════════════════════════════════════════
# SECTION 6: DEMO WITH SAMPLE DOCUMENTS
# ═══════════════════════════════════════════════════════════

# Sample knowledge base documents
SAMPLE_DOCS = {
    "transformer_architecture": """
The Transformer architecture was introduced in the paper "Attention Is All You Need"
by Vaswani et al. in 2017. It revolutionized natural language processing by replacin
recurrent neural networks with self-attention mechanisms.

The key innovation is the self-attention mechanism, which allows the model to weigh
the importance of different parts of the input when processing each element. This
enables parallel processing of all input tokens, making training much faster than
sequential models like LSTMs.

The architecture consists of an encoder and decoder, each composed of multiple layer
Each layer contains multi-head self-attention and feed-forward neural networks, with
residual connections and layer normalization.

Modern large language models like GPT-4 and Llama use a decoder-only variant of the
Transformer. They process text autoregressively, predicting one token at a time base
on all previous tokens.
""",

    "rlhf_training": """
Reinforcement Learning from Human Feedback (RLHF) is a technique used to align
language models with human preferences. It was popularized by OpenAI and Anthropic
for training models like ChatGPT and Claude.

The RLHF process has three main steps:

1. Supervised Fine-Tuning (SFT): The base model is fine-tuned on a dataset of
   high-quality (instruction, response) pairs to learn instruction-following behavio

2. Reward Model Training: Human evaluators rank multiple model outputs for the same
   prompt. These rankings train a reward model that predicts human preferences.

3. Policy Optimization: The language model is further trained using reinforcement
   learning (specifically PPO) to maximize the reward model's score, with a KL
   divergence penalty to prevent deviating too far from the SFT model.

DPO (Direct Preference Optimization) is an alternative to RLHF that skips the
reward model entirely. It directly optimizes the policy using preference data,
which is simpler and more stable than the full RLHF pipeline.
""",
```

```
    "rag_systems": """
Retrieval-Augmented Generation (RAG) is a technique that enhances language models
by allowing them to retrieve relevant information from external knowledge bases
before generating responses.

The RAG pipeline consists of several key components:

Chunking: Documents are split into smaller pieces (chunks) that can be individually
embedded and retrieved. Common strategies include fixed-size chunking, recursive
splitting by paragraph/sentence boundaries, and semantic chunking based on topic
changes.

Embedding: Each chunk is converted into a dense vector representation using an
embedding model. Popular models include OpenAI's text-embedding-3-small and
open-source alternatives like all-MiniLM-L6-v2.

Vector Storage: Embeddings are stored in a vector database that supports efficient
similarity search. Options include ChromaDB, Pinecone, FAISS, and pgvector.

Retrieval: When a user asks a question, the question is embedded and compared
against stored chunk embeddings using cosine similarity. The top-K most similar
chunks are retrieved.

Hybrid Search: Combining vector search with keyword-based search (BM25) often
produces better results than either method alone. Reciprocal Rank Fusion (RRF)
is a popular method for combining multiple ranking lists.

Context Window Management: Retrieved chunks must fit within the LLM's context
window along with the system prompt and user query. Strategies include limiting
the number of chunks, reranking, and summarization.
""",

    "agent_architectures": """
AI agents are systems that use language models to autonomously plan, reason,
and take actions to accomplish tasks. Several key architectures have emerged:

ReAct (Reason + Act): The agent alternates between thinking (reasoning about
what to do) and acting (using tools). Each observation from a tool informs
the next reasoning step. This is effective for tasks requiring real-time
information gathering and step-by-step problem solving.

Plan-then-Execute: A planner generates a complete plan upfront, then an
executor carries out each step. A critic validates results after each step,
enabling retries and replanning. Better for complex multi-step tasks with
dependencies between steps.

Multi-Agent Systems: Multiple specialized agents collaborate on a task.
Common patterns include a Planner agent (decomposes tasks), Executor agent
(runs tools), Researcher agent (gathers information), and Critic agent
(validates quality). An orchestrator coordinates the agents.

Agent Memory: Effective agents use multiple memory types. Short-term memory
(the conversation scratchpad) handles the current task. Long-term memory
(vector stores, RAG) provides persistent knowledge. Episodic memory logs
past actions and outcomes for learning from experience.
```

```
    Common failure modes include infinite loops (same action repeated),
    hallucinated tool calls (inventing non-existent tools), and premature
    termination (answering before having sufficient information).
    """
    }


    def demo():
        """Run the RAG pipeline demo."""
        # Initialize pipeline
        rag = RAGPipeline(chunk_size=400, chunk_overlap=50, top_k=3)

        # Ingest documents
        rag.ingest(SAMPLE_DOCS)

        # Test queries
        print("\n" + "="*60)
        print("QUERY 1: Factual retrieval")
        print("="*60)
        rag.query("What is the Transformer architecture and who created it?")

        print("\n" + "="*60)
        print("QUERY 2: Technical details")
        print("="*60)
        rag.query("How does RLHF work and what are its three steps?")

        print("\n" + "="*60)
        print("QUERY 3: Comparison")
        print("="*60)
        rag.query("What is the difference between ReAct and Plan-then-Execute?")

        print("\n" + "="*60)
        print("QUERY 4: RAG about RAG")
        print("="*60)
        rag.query("What are the components of a RAG pipeline?")

        # Demonstrate dynamic memory
        print("\n" + "="*60)
        print("ADDING DYNAMIC MEMORY")
        print("="*60)
        rag.add_memory(
            "team_decision",
            "Our team decided to use ChromaDB for the vector store because "
            "it's easy to set up, supports persistent storage, and has good "
            "Python integration. We chose text-embedding-3-small from OpenAI "
            "for embeddings due to its balance of quality and cost."
        )

        print("\n" + "="*60)
        print("QUERY 5: Using dynamic memory")
        print("="*60)
        rag.query("What vector database did our team decide to use and why?")

        # Show out-of-scope handling
        print("\n" + "="*60)
        print("QUERY 6: Out of scope")
```

```
    print("="*60)
    rag.query("What is the capital of France?")


if __name__ == "__main__":
    demo()
```

Using ChromaDB (Production Alternative)

```
"""
Production RAG with ChromaDB — use this when you need persistent,
performant vector search.

pip install chromadb openai
"""
import chromadb
from chromadb.utils import embedding_functions
import openai

# Initialize ChromaDB
client = chromadb.PersistentClient(path="./chroma_db")

# Use OpenAI embeddings (or switch to local)
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key="your-api-key",
    model_name="text-embedding-3-small"
)

# Create a collection
collection = client.get_or_create_collection(
    name="knowledge_base",
    embedding_function=openai_ef,
    metadata={"hnsw:space": "cosine"}  # Use cosine similarity
)

# Add documents
collection.add(
    documents=["The Transformer architecture was introduced in 2017...",
               "RLHF is used to align language models..."],
    metadatas=[{"source": "transformers.md"}, {"source": "rlhf.md"}],
    ids=["doc1", "doc2"]
)

# Query
results = collection.query(
    query_texts=["How does attention work?"],
    n_results=3,
    include=["documents", "metadatas", "distances"]
)

print(f"Top result: {results['documents'][0][0][:100]}...")
print(f"Distance: {results['distances'][0][0]:.4f}")
```

```
print(f"Source: {results['metadatas'][0][0]['source']}")

# Use in RAG prompt
context = "\n\n".join(results["documents"][0])
prompt = f"""Answer based on this context:

{context}

Question: How does attention work?
Answer:"""

response = openai.chat.completions.create(
    model="gpt-4",
    messages=[{"role": "user", "content": prompt}]
)
print(response.choices[0].message.content)
```

✓ Day 5 Deliverable Checklist

- [ ] Built a complete RAG pipeline (chunker + embedder + vector store)
- [ ] Implemented BM25 keyword search
- [ ] Combined vector + BM25 into hybrid search
- [ ] Added a grounding score evaluator
- [ ] Agent can store new memories dynamically
- [ ] Agent can retrieve relevant context for questions
- [ ] Tested with at least 4 different queries
- [ ] Can explain: What is RAG and when should you use it vs fine-tuning?
- [ ] Can explain: How does cosine similarity work?
- [ ] Can explain: Why is hybrid search better than pure vector search?
- [ ] Can explain: What are chunking strategies and their trade-offs?

Next → Day 6: Multi-Agent Collaboration

Appendix: RAG Products Reference

A comprehensive guide to the major products in the RAG ecosystem — what they do, how they work under the hood, best practices, and where they fall short.

A.1 Vector Databases

Vector databases are the **storage and retrieval backbone** of RAG. They index embeddings and serve nearest-neighbor queries at scale.

ChromaDB

```
Type:        Embedded / Client-Server vector database
Language:    Python (core), Rust (planned)
ANN Index:   HNSW (via hnswlib)
License:     Apache 2.0 (open source)
Best For:    Prototyping, small-to-medium production (< 10M vectors)

Architecture:
  ┌─────────────────────────────────────────┐
  │  Your Python App                        │
  │  ┌──────────────────────────────────────┐│
  │  │ ChromaDB (embedded mode)             ||
  │  │  ┌────────────┐  ┌──────────────────┐ ||
  │  │  │ HNSW       │  │ Metadata Store   │ ||
  │  │  │ Index      │  │ (SQLite/DuckDB)  │ ||
  │  │  └────────────┘  └──────────────────┘ ||
  │  └──────────────────────────────────────┘|
  └─────────────────────────────────────────┘

  OR: Client-Server mode (HTTP API on port 8000)

Best Practices:
  ✅ Use persistent client for production (PersistentClient)
  ✅ Set collection metadata: {"hnsw:space": "cosine"}
  ✅ Include rich metadata for filtering (source, date, category)
  ✅ Use built-in embedding functions or bring your own
  ✅ Batch upserts (add documents in groups of 100-500)

Limitations:
  ❌ No built-in horizontal scaling (single-node only)
  ❌ No built-in replication or high availability
  ❌ Limited query language compared to dedicated DBs
  ❌ HNSW index rebuild on large updates can be slow
  ❌ Not ideal for > 10M vectors (memory-bound)

When to Choose:
  → You're building a prototype or MVP
  → You want zero-config, pip install and go
  → Dataset is small-to-medium (< 5M vectors)
  → You're already in Python and want embedded mode
```

Pinecone

```
Type:        Fully managed vector database (SaaS)
ANN Index:   Proprietary (HNSW-based with optimizations)
License:     Proprietary (free tier available)
```

```
Best For:    Production workloads without infra management

Architecture:
  ┌──────────┐      HTTPS      ┌──────────────────────────┐
  │ Your App ├─────────────────→│ Pinecone Cloud           │
  └──────────┘                 │  ┌──────────────────┐    │
                               │  │ Index Pods/      │    │
                               │  │ Serverless       │    │
                               │  │ (auto-scaling)   │    │
                               │  └──────────────────┘    │
                               │  ┌──────────────────┐    │
                               │  │ Metadata Store   │    │
                               │  └──────────────────┘    │
                               └──────────────────────────┘

  Two deployment modes:
    Pods:       Dedicated hardware, predictable performance
    Serverless: Pay-per-query, auto-scales, lower cost for spiky traffic

Best Practices:
  ✅ Use serverless for most new projects (simpler + cheaper)
  ✅ Use namespaces to organize data (multi-tenant, by category)
  ✅ Keep vector dimensions ≤ 1536 for optimal performance
  ✅ Use metadata filtering to narrow search before ANN
  ✅ Batch upserts (up to 100 vectors per request)
  ✅ Use sparse-dense vectors for hybrid search

Limitations:
  ❌ Vendor lock-in (proprietary, can't self-host)
  ❌ Data leaves your infrastructure (compliance concern)
  ❌ Cost grows significantly at scale (pay per vector stored + queries)
  ❌ Limited query flexibility vs. a full database
  ❌ Cold starts on serverless for rarely-accessed namespaces
  ❌ 20K dimension limit, 40KB metadata limit per vector

When to Choose:
  → You don't want to manage infrastructure
  → You need production-grade reliability without DevOps
  → Budget is flexible and dataset is moderate (< 100M vectors)
  → You need multi-region or global distribution
```

Qdrant

```
Type:        Open-source vector database (self-hosted or cloud)
Language:    Rust (high performance)
ANN Index:   HNSW with custom optimizations + quantization
License:     Apache 2.0
Best For:    Production self-hosted, performance-critical workloads

Architecture:
  ┌──────────────────────────────────────────────┐
  │  Qdrant Cluster                              │
  │  ┌────────────┐ ┌────────────┐ ┌────────────┐ │
  │  │  Shard 1   │ │  Shard 2   │ │  Shard 3   │ │
```

```
  │  ┌────────┐  │ ┌────────┐  │ ┌────────┐  │   │
  │  │ HNSW   │  │ │ HNSW   │  │ │ HNSW   │  │   │
  │  └────────┘  │ └────────┘  │ └────────┘  │   │
  │  Replicas    │ Replicas    │ Replicas    │   │
  │  └──────────┘ └──────────┘ └──────────┘   │
  │  gRPC / REST API                          │
  └──────────────────────────────────────────┘

  Quantization options:
    Scalar:  float32 → int8 (4× memory savings, ~1% recall loss)
    Product: PQ compression (up to 64× savings)
    Binary:  float32 → 1 bit (32× savings, use for oversampling+rerank)

Best Practices:
  ✅ Use scalar quantization for best speed/recall tradeoff
  ✅ Enable WAL (Write-Ahead Log) for durability
  ✅ Use payload indexing for frequently filtered fields
  ✅ Set optimizers for your workload (indexing_threshold, memmap)
  ✅ Use gRPC client for lower latency (vs REST)
  ✅ Use collection aliases for zero-downtime reindexing

Limitations:
  ❌ More operational complexity than managed solutions
  ❌ Cluster setup requires understanding of sharding/replication
  ❌ No built-in embedding generation (bring your own)
  ❌ Smaller ecosystem/community than Pinecone or Weaviate

When to Choose:
  → You need self-hosted with full control
  → Performance and low latency are critical
  → You want open-source with no vendor lock-in
  → You need advanced filtering (nested, geo, range queries)
```

Weaviate

```
Type:        Open-source vector database with built-in ML modules
Language:    Go
ANN Index:   HNSW + PQ (flat index for small collections)
License:     BSD-3-Clause
Best For:    Multi-modal search, built-in vectorization

Architecture:
  ┌──────────────────────────────────────────────┐
  │  Weaviate                                    │
  │  ┌──────────────┐ ┌──────────────┐ ┌──────────────┐ │
  │  │ HNSW Index   │ │ Object Store │ │ Inverted     │ │
  │  │              │ │ (properties) │ │ Index (BM25) │ │
  │  └──────────────┘ └──────────────┘ └──────────────┘ │
  │  ┌──────────────────────────────────────────┐ │
  │  │ ML Module Layer (pluggable)              │ │
  │  │  text2vec-openai | img2vec-neural | multi2vec │ │
  │  └──────────────────────────────────────────┘ │
  │  GraphQL + REST API                          │
  └──────────────────────────────────────────┘
```

```
  Unique: Built-in vectorizer modules — pass raw text/images,
  Weaviate generates embeddings automatically.

Best Practices:
  ✅ Use vectorizer modules to avoid managing embedding pipelines
  ✅ Enable hybrid search (vector + BM25) with autocut
  ✅ Use generative modules for RAG built into queries
  ✅ Define schema with proper data types for filtering
  ✅ Use PQ compression for large datasets (> 1M objects)
  ✅ Use multi-tenancy for SaaS apps (isolation per tenant)

Limitations:
  ❌ Heavier resource usage than simpler DBs (Go + ML modules)
  ❌ GraphQL query syntax has a learning curve
  ❌ Module dependency management adds complexity
  ❌ Slower cold starts due to ML module loading
  ❌ Write throughput lower than Qdrant for batch imports

When to Choose:
  → You want built-in embedding generation (no separate pipeline)
  → You need multi-modal search (text + images + video)
  → You want RAG generation built into the database layer
  → Your team prefers GraphQL over REST
```

pgvector (PostgreSQL Extension)

```
Type:        PostgreSQL extension for vector similarity search
Language:    C
ANN Index:   HNSW + IVFFlat
License:     PostgreSQL License (open source)
Best For:    Teams already using PostgreSQL, unified data layer

Architecture:
  ┌──────────────────────────────────────────────────┐
  │  PostgreSQL                                      │
  │  ┌────────────────────────────────────────────┐  │
  │  │ Your existing tables (users, products, docs...)  │ │
  │  ├────────────────────────────────────────────┤  │
  │  │ pgvector extension                         │  │
  │  │  ┌──────────┐  ┌──────────┐                │  │
  │  │  │ HNSW     │  │ IVFFlat  │                │  │
  │  │  │ Index    │  │ Index    │                │  │
  │  │  └──────────┘  └──────────┘                │  │
  │  │  vector(1536) column type                  │  │
  │  │  <=> (cosine)  <#> (inner product)  <-> (L2)  │ │
  │  └────────────────────────────────────────────┘  │
  │  Standard SQL                                    │
  └──────────────────────────────────────────────────┘

  Key advantage: Vectors live alongside your relational data.
  JOIN vectors with users, filter by SQL WHERE, use transactions.

Best Practices:
```

```
  ✅ Use HNSW index (not IVFFlat) — better recall, no training needed
  ✅ Set appropriate m and ef_construction for HNSW
     CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops)
       WITH (m = 16, ef_construction = 64);
  ✅ Use partial indexes for filtered searches
  ✅ SET hnsw.ef_search = 100; for better recall at query time
  ✅ Normalize vectors before insert if using inner product
  ✅ Use EXPLAIN ANALYZE to verify index is being used

Limitations:
  ❌ Slower than purpose-built vector DBs (Qdrant, Weaviate)
  ❌ HNSW index build is memory-intensive (SET maintenance_work_mem)
  ❌ No built-in sharding for vectors (relies on Citus or partitioning)
  ❌ Limited to ~10M vectors per index on typical hardware
  ❌ No native hybrid search (need pg_trgm or ParadeDB for BM25)
  ❌ No built-in embedding generation

When to Choose:
  → You already run PostgreSQL and want to avoid another DB
  → You need JOINs between vectors and relational data
  → You want ACID transactions on vector operations
  → Dataset is moderate (< 5-10M vectors)
  → You want a unified backup/restore strategy
```

FAISS (Facebook AI Similarity Search)

```
Type:        Library (not a database — no persistence, no server)
Language:    C++ with Python bindings
ANN Index:   IVF, PQ, IVF-PQ, HNSW, Flat (brute force), and more
License:     MIT
Best For:    Research, custom pipelines, maximum flexibility

Architecture:
  ┌──────────────────────────────────────────────────┐
  │  Your Application                                │
  │  ┌──────────────────────────────────────────────┐ │
  │  │ FAISS (in-process library)                   │ │
  │  │                                              │ │
  │  │  Index types:                                │ │
  │  │    IndexFlatL2        — brute force (exact)  │ │
  │  │    IndexIVFFlat       — IVF clustering       │ │
  │  │    IndexIVFPQ         — IVF + Product Quantiz.│ │
  │  │    IndexHNSWFlat      — HNSW graph           │ │
  │  │    IndexScalarQuantizer — scalar compression │ │
  │  │                                              │ │
  │  │  GPU acceleration: faiss-gpu (CUDA)          │ │
  │  └──────────────────────────────────────────────┘ │
  │                                                  │
  │  ⚠ No server, no REST API, no persistence by default │
  │    (use faiss.write_index / faiss.read_index to save) │
  └──────────────────────────────────────────────────┘

  String index factory — compose indexes declaratively:
    "IVF4096,PQ64"    → IVF with 4096 cells + PQ with 64 sub-vectors
```

```
    "HNSW32"          → HNSW with M=32
    "OPQ16,IVF1024,PQ16" → OPQ rotation + IVF + PQ

Best Practices:
  ✅ Use index factory strings for readability
  ✅ Train on a representative sample (not the full dataset)
  ✅ Use GPU indexes for training speed (faiss-gpu)
  ✅ Use IVF-PQ for billion-scale, HNSW for < 50M
  ✅ Tune nprobe at query time (start with 10, increase for recall)
  ✅ Use OPQ (Optimized PQ) rotation for better PQ accuracy
  ✅ Wrap in a server if needed (use with LangChain or build custom)

Limitations:
  ❌ Not a database — no metadata storage, no filtering, no CRUD
  ❌ No built-in server or API (must wrap yourself)
  ❌ Requires manual persistence (write_index/read_index)
  ❌ Complex API — many index types, hard to choose
  ❌ IVF requires training step before adding vectors
  ❌ Python GIL can bottleneck multi-threaded queries

When to Choose:
  → You need maximum control over indexing algorithms
  → You're doing ML research or benchmarking
  → You have GPU resources for billion-scale indexing
  → You're building a custom search pipeline
  → You need algorithms not available in other DBs
```

Milvus

```
Type:        Distributed open-source vector database
Language:    Go + C++
ANN Index:   IVF, HNSW, DiskANN, GPU indexes, SCANN
License:     Apache 2.0
Best For:    Billion-scale production, distributed workloads

Architecture:
  ┌──────────────────────────────────────────────────────────┐
  │  Milvus Cluster                                          │
  │                                                          │
  │  ┌──────────┐  ┌──────────────┐  ┌──────────────────┐    │
  │  │ Proxy    │  │ Query Nodes  │  │ Data Nodes       │    │
  │  │ (API)    │  │ (search)     │  │ (insert/persist) │    │
  │  └──────────┘  └──────────────┘  └──────────────────┘    │
  │  ┌──────────────────────────────────────────────────┐    │
  │  │ Storage: S3/MinIO + etcd (metadata) + Pulsar/    │    │
  │  │          Kafka (log/stream)                      │    │
  │  └──────────────────────────────────────────────────┘    │
  └──────────────────────────────────────────────────────────┘


  Also available as:
    Milvus Lite  — embedded (like ChromaDB, for prototyping)
    Zilliz Cloud — fully managed SaaS version

Best Practices:
```

```
  ✅ Use Milvus Lite for development, cluster for production
  ✅ Choose index by scale: HNSW (< 50M), DiskANN (> 50M)
  ✅ Use partition keys for multi-tenant data isolation
  ✅ Enable dynamic schema for flexible metadata
  ✅ Use GPU indexes (IVF_FLAT on GPU) for highest throughput
  ✅ Tune segment size for batch insert performance

Limitations:
  ❌ Complex deployment (many components: etcd, MinIO, Pulsar)
  ❌ Steep learning curve for cluster configuration
  ❌ Higher operational overhead than single-node DBs
  ❌ Resource-heavy — needs substantial RAM/CPU for cluster
  ❌ Community edition lacks some enterprise features

When to Choose:
  → You're operating at billion-vector scale
  → You need true distributed, horizontally scalable search
  → You have the DevOps capacity to run a complex cluster
  → You need GPU-accelerated indexing and search
```

A.1b Azure Vector & Search Services

Azure provides a fully integrated RAG ecosystem. If you're already on Azure, these services offer the tightest integration with Azure OpenAI, Entra ID (auth), and enterprise compliance.

Azure AI Search (formerly Azure Cognitive Search)

```
Type:        Fully managed search service (SaaS) with vector support
ANN Index:   HNSW (default), Exhaustive KNN (brute force)
License:     Proprietary (Azure service)
Best For:    Enterprise RAG on Azure, hybrid search, integrated AI pipelines

Architecture:
  ┌──────────────────────────────────────────────────────────┐
  │  Azure AI Search                                         │
  │                                                          │
  │  ┌──────────────────────────────────────────────────┐    │
  │  │ Search Index                                     │    │
  │  │  ┌──────────┐  ┌──────────────┐  ┌────────────┐  │    │
  │  │  │ Vector   │  │ Full-Text    │  │ Semantic   │  │    │
  │  │  │ Fields   │  │ Index (BM25) │  │ Ranker (L2)│  │    │
  │  │  │ (HNSW)   │  │ (Lucene)     │  │ (cross-enc)│  │    │
  │  │  └──────────┘  └──────────────┘  └────────────┘  │    │
  │  └──────────────────────────────────────────────────┘    │
  │                                                          │
  │  ┌──────────────────────────────────────────────────┐    │
  │  │ Integrated AI Enrichment Pipeline (Skillsets)    │    │
  │  │  OCR → Key Phrases → Entity Recognition → Embeddings │
  │  │  (connects to Azure AI Services & Azure OpenAI)  │    │
  │  └──────────────────────────────────────────────────┘    │
  │                                                          │
```

```
  │  ┌──────────────────────────────────────────────────────┐ │
  │  │ Indexers (auto-sync from data sources)               │ │
  │  │  Blob Storage │ Cosmos DB │ SQL DB │ Data Lake │ etc.  │ │
  │  └──────────────────────────────────────────────────────┘ │
  │                                                          │
  │  REST API + SDKs (.NET, Python, Java, JavaScript)        │
  └──────────────────────────────────────────────────────────┘

  Three search modes in a SINGLE query:
    1. Vector search     — ANN via HNSW
    2. Full-text search  — BM25 via Lucene
    3. Semantic ranking  — built-in cross-encoder reranker (L2)
    → Hybrid = all three combined automatically

  Integrated vectorization (preview):
    Upload raw text → Azure AI Search calls Azure OpenAI to embed
    → vectors stored automatically. No external embedding pipeline needed.

Best Practices:
  ✅ Enable Semantic Ranker (built-in reranking, huge quality boost)
  ✅ Use hybrid search (vector + full-text + semantic) by default
  ✅ Use integrated vectorization to simplify the embedding pipeline
  ✅ Use Indexers to auto-sync from Blob Storage, Cosmos DB, SQL
  ✅ Use Skillsets for AI enrichment (OCR, entity extraction, etc.)
  ✅ Use vector compression (scalar/binary quantization) to reduce cost
  ✅ Use "On Your Data" feature with Azure OpenAI for zero-code RAG
  ✅ Restrict access with Azure RBAC and Managed Identities
  ✅ Use index aliases for zero-downtime reindexing

Limitations:
  ❌ More expensive than open-source alternatives at scale
  ❌ Azure lock-in (not portable to other clouds)
  ❌ Max 3072 dimensions per vector field
  ❌ Semantic Ranker adds latency (~200-500ms) and extra cost
  ❌ Limited to 1000 results per query (pagination ceiling)
  ❌ Complex pricing model (per-unit, per-query, per-enrichment)
  ❌ Index schema changes may require rebuild

When to Choose:
  → You're already on Azure (tightest integration)
  → You need hybrid search + built-in reranking out of the box
  → You want auto-sync from Azure data sources (Blob, Cosmos, SQL)
  → Enterprise compliance requirements (SOC 2, HIPAA, Entra ID)
  → You want "On Your Data" zero-code RAG with Azure OpenAI
  → Your team is .NET-first or Java-first
```

Azure Cosmos DB (Vector Search)

```
Type:        Globally distributed multi-model database with vector support
ANN Index:   DiskANN (default), Flat (brute force)
License:     Proprietary (Azure service)
Best For:    Global apps needing vectors + transactional data together

Architecture:
```

```
┌──────────────────────────────────────────────────────────┐
│  Azure Cosmos DB                                         │
│                                                          │
│  ┌──────────────────────────────────────────────────┐    │
│  │ Container (collection)                           │    │
│  │  ┌──────────────────┐  ┌──────────────────────┐  │    │
│  │  │ Document Store   │  │ Vector Index (DiskANN)│  │    │
│  │  │ (JSON documents) │  │ embedded in same item │  │    │
│  │  └──────────────────┘  └──────────────────────┘  │    │
│  └──────────────────────────────────────────────────┘    │
│                                                          │
│  Global distribution: multi-region read/write replicas   │
│  APIs: NoSQL, MongoDB, PostgreSQL (pgvector), Cassandra  │
│  Consistency levels: Strong → Eventual (5 options)       │
└──────────────────────────────────────────────────────────┘

  Two vector-capable APIs:
    NoSQL API:    Native vector search with DiskANN index
    PostgreSQL API (vCore): pgvector extension, familiar SQL

Best Practices:
  ✅ Use NoSQL API + DiskANN for best vector performance
  ✅ Co-locate vectors with related data (same document/item)
  ✅ Use composite indexes for combined vector + filter queries
  ✅ Choose consistency level per use case (session for RAG is fine)
  ✅ Use PostgreSQL vCore API if your team knows pgvector/SQL
  ✅ Leverage global distribution for low-latency multi-region RAG
  ✅ Use hierarchical partitioning for multi-tenant vector isolation

Limitations:
  ❌ More expensive than dedicated vector DBs for pure vector workloads
  ❌ DiskANN recall may be lower than HNSW for small datasets
  ❌ Vector dimensions limited (NoSQL: 4096, PostgreSQL: 2000)
  ❌ Not as many vector-specific features as Qdrant/Weaviate
  ❌ RU-based pricing can be hard to predict for vector queries
  ❌ No built-in embedding generation

When to Choose:
  → You already use Cosmos DB for your application data
  → You need vectors + transactional data in the same database
  → You need global distribution with multi-region low latency
  → You want to avoid managing a separate vector database
  → You need enterprise SLAs (99.999% availability)
```

A.2 RAG Orchestration Frameworks

These frameworks wire together the RAG pipeline: loading → chunking → embedding → storing → retrieving → prompting.

LangChain

```
Type:        RAG / LLM orchestration framework
Language:    Python, JavaScript/TypeScript
License:     MIT
Best For:    Rapid prototyping, widest integration ecosystem

What It Provides:
  ┌──────────────────────────────────────────────────┐
  │  LangChain                                       │
  │                                                  │
  │  Document Loaders:  PDF, HTML, Notion, S3, GCS, ...  │
  │  Text Splitters:    Recursive, Token, Markdown, ...  │
  │  Embeddings:        OpenAI, Cohere, HuggingFace, ... │
  │  Vector Stores:     ChromaDB, Pinecone, pgvector, ...│
  │  Retrievers:        Vector, BM25, Hybrid, Multi-query│
  │  Chains/Agents:     RetrievalQA, ConversationalRAG   │
  │                                                  │
  │  LangSmith:  Tracing, evaluation, monitoring     │
  │  LangServe:  Deploy chains as REST APIs          │
  │  LangGraph:  Stateful multi-step agent workflows │
  └──────────────────────────────────────────────────┘

Best Practices:
  ✅ Use LCEL (LangChain Expression Language) for composability
  ✅ Use LangSmith for debugging retrieval quality
  ✅ Use MultiQueryRetriever — generates query variations for better recall
  ✅ Use ParentDocumentRetriever for small-chunk-embed, full-doc-retrieve
  ✅ Use EnsembleRetriever for hybrid search (BM25 + vector)
  ✅ Pin dependency versions — API changes frequently

Limitations:
  ❌ Heavy abstraction — can obscure what's actually happening
  ❌ Frequent breaking API changes between versions
  ❌ Debugging through abstraction layers is frustrating
  ❌ Performance overhead from abstraction layers
  ❌ "Chain" paradigm can be limiting for complex flows (→ use LangGraph)
  ❌ Large dependency tree, bloated for simple use cases

When to Choose:
  → You want the fastest path from zero to working RAG
  → You need to swap components easily (try different DBs, models)
  → You want observability via LangSmith
  → You're building complex agent workflows (LangGraph)
```

LlamaIndex

```
Type:        Data framework for LLM applications
Language:    Python, TypeScript
License:     MIT
Best For:    Structured data ingestion, advanced retrieval strategies

What It Provides:
  ┌──────────────────────────────────────────────────┐
  │  LlamaIndex                                      │
```

```
   │                                                    │
   │  Data Connectors:  130+ sources (Notion, Slack, DB)  │
   │  Index Types:                                      │
   │    VectorStoreIndex  — standard vector RAG         │
   │    TreeIndex         — hierarchical summarization  │
   │    KeywordTableIndex — keyword-based retrieval     │
   │    KnowledgeGraphIndex — entity/relation graphs    │
   │  Query Engine:       Routing, sub-questions, SQL   │
   │  Response Synthesis: Refine, compact, tree_summarize  │
   │                                                    │
   │  LlamaCloud:  Managed parsing + indexing service   │
   │  LlamaParse:  PDF/doc parsing with layout awareness  │
   └────────────────────────────────────────────────────┘

Best Practices:
  ✅ Use VectorStoreIndex for standard RAG (simplest path)
  ✅ Use SubQuestionQueryEngine to decompose complex queries
  ✅ Use SentenceWindowNodeParser for better chunk context
  ✅ Use ResponseSynthesizer with mode="compact" for efficiency
  ✅ Use LlamaParse for complex PDFs (tables, images, layouts)
  ✅ Use node postprocessors for reranking and filtering

Limitations:
  ❌ Steeper learning curve than LangChain (more concepts)
  ❌ Smaller community and fewer tutorials
  ❌ Less flexible for non-RAG agent patterns
  ❌ Index abstractions can hide performance issues
  ❌ Some advanced features require LlamaCloud (paid)

When to Choose:
  → Your data is complex (PDFs with tables, structured docs)
  → You need advanced retrieval (hierarchical, knowledge graph)
  → You want built-in query decomposition
  → You need to index structured data (SQL tables, Pandas DFs)
```

Semantic Kernel (Microsoft)

```
Type:        AI orchestration SDK for enterprise applications
Language:    C# (.NET), Python, Java
License:     MIT
Best For:    .NET-first teams, Azure-native RAG, enterprise C# apps

What It Provides:
  ┌────────────────────────────────────────────────────┐
  │  Semantic Kernel                                   │
  │                                                    │
  │  Connectors:                                       │
  │    AI Services:  Azure OpenAI, OpenAI, HuggingFace   │
  │    Memory:       Azure AI Search, Qdrant, Chroma,  │
  │                  Cosmos DB, pgvector, Pinecone     │
  │                                                    │
  │  Core Abstractions:                                │
  │    Kernel       → central orchestrator             │
  │    Plugins      → reusable functions (native + AI)   │
```

```
   │    Memory       → vector store abstraction layer     │
   │    Planner      → auto-compose plugins into plans    │
   │    Filters      → middleware for logging, auth, etc. │
   │                                                      │
   │  RAG Pattern:                                        │
   │    TextMemoryPlugin → embed, store, recall context   │
   │    VectorStoreRecordCollection → typed vector CRUD   │
   │    SearchPlugin → integrate Azure AI Search          │
   └──────────────────────────────────────────────────────┘

  Deep Azure integration:
    Azure OpenAI  → LLM + embeddings (same Entra ID auth)
    Azure AI Search → vector + hybrid + semantic ranking
    Cosmos DB     → vector memory with global distribution
    Managed Identity → no API keys in code

Best Practices:
  ✅ Use IVectorStore abstraction for portable vector store code
  ✅ Use Azure OpenAI connector with Managed Identity (no keys)
  ✅ Combine Semantic Kernel plugins with Azure AI Search
  ✅ Use Kernel Filters for logging, telemetry, and guardrails
  ✅ Use DI (dependency injection) to register Kernel in ASP.NET
  ✅ Use Process Framework for multi-step RAG workflows

Limitations:
  ❌ Smaller community than LangChain / LlamaIndex
  ❌ Python/Java SDKs lag behind C# in features
  ❌ Less tutorial content and fewer examples online
  ❌ Planner can be unpredictable for complex plans
  ❌ Fewer data connectors/loaders than LangChain

When to Choose:
  → You're a .NET / C# team building on Azure
  → You want first-class Azure OpenAI + Azure AI Search integration
  → You need enterprise patterns (DI, typed models, middleware)
  → You want an SDK backed by Microsoft long-term
  → You're building agents with Semantic Kernel Process Framework
```

A.3 Embedding Models

The model that converts text → vectors. Critical choice — determines retrieval quality.

Model	Dims	Max Tokens	Notes
Azure OpenAI text-embedding-3-large	3072 (or less)	8191	Same models as OpenAI, hosted on Azure. Entra ID auth, VNet, data residency.
Azure OpenAI text-embedding-3-small	1536	8191	Enterprise-ready. Private endpoints, no data logging.

OpenAI text-embedding-3-large	3072 (or less)	8191	Best quality (API). Supports dimension reduction (e.g., 1536).
OpenAI text-embedding-3-small	1536	8191	Good quality, lower cost. Best cost/quality ratio.
Cohere embed-v3	1024	512	Multilingual. Supports search_document/query types.
BGE-large-en-v1.5 (BAAI)	1024	512	Best open-source (English). Self-hosted, free.
BGE-M3 (BAAI)	1024	8192	Multilingual + multi-granularity. Sparse+Dense.
Jina embeddings-v3	1024	8192	Long context. Good for documents, code, and more.
all-MiniLM-L6-v2 (sentence-transformers)	384	256	Tiny, fast. Good for dev and edge deployment.

Best Practices:

- ✅ Match embedding model dimensions to your vector DB config
- ✅ Use the SAME model for indexing and querying (critical!)
- ✅ Use instruction-prefixed models correctly:
  Index: "passage: How RLHF improves alignment..."
  Query: "query: What is RLHF?"
- ✅ Benchmark on YOUR data — MTEB leaderboard ≠ your domain
- ✅ Consider fine-tuning embeddings for domain-specific data
- ✅ Use dimensionality reduction (text-embedding-3) to save memory

Limitations to Know:

- ❌ API models (OpenAI, Cohere) → vendor dependency, cost, latency
- ❌ Open models → need GPU for fast throughput, quality may lag
- ❌ All models have max token limits — long docs need chunking
- ❌ Embedding models are NOT multilingual by default (except BGE-M3)
- ❌ Changing embedding model requires full re-indexing

A.4 Document Parsing & Loading

Getting data **into** the pipeline is often the hardest part of RAG.

Tool	What It Does
Azure Document Intelligence (formerly Form Recognizer)	Azure AI service for document parsing. Handles PDFs, images, Office docs, invoices. Best-in-class table extraction & OCR. Prebuilt models for invoices, receipts, IDs. Custom models for domain-specific docs.

```
│                      │ Layout model outputs Markdown with tables.    │
│                      │ ⚠ Azure service (pay per page).               │
├──────────────────────┼──────────────────────────────────────────────┤
│ LlamaParse           │ AI-powered PDF/doc parser. Handles tables,    │
│                      │ images, complex layouts. Best quality.        │
│                      │ ⚠ Paid API. Use for complex documents.        │
├──────────────────────┼──────────────────────────────────────────────┤
│ Unstructured.io      │ Open-source document parsing library.         │
│                      │ Supports PDF, DOCX, PPTX, HTML, Markdown.     │
│                      │ Good layout detection. Heavy dependencies.    │
├──────────────────────┼──────────────────────────────────────────────┤
│ PyPDF / PyMuPDF      │ Basic PDF text extraction. Fast, lightweight. │
│                      │ Struggles with tables, images, scanned PDFs.  │
├──────────────────────┼──────────────────────────────────────────────┤
│ Docling (IBM)        │ Open-source, AI-powered doc understanding.    │
│                      │ Handles tables, figures, document structure.  │
│                      │ Export to Markdown, JSON. Good quality.       │
├──────────────────────┼──────────────────────────────────────────────┤
│ Apache Tika          │ Java-based parser for 1000+ file formats.     │
│                      │ Reliable but basic extraction quality.        │
└──────────────────────┴──────────────────────────────────────────────┘

Best Practices:
  ✅ Test your parser on YOUR documents (quality varies wildly)
  ✅ Use layout-aware parsing for PDFs with tables
  ✅ Preserve document structure (headings, sections) for chunking
  ✅ Handle OCR separately for scanned documents (Tesseract, AWS Textract)
  ✅ Clean extracted text (remove headers/footers, fix encoding)
  ✅ Azure users: Azure Doc Intelligence Layout model → Markdown output
     feeds directly into Azure AI Search indexers via skillsets
```

A.5 End-to-End RAG Best Practices

```
┌──────────────────────────────────────────────────────────────────┐
│                  THE RAG QUALITY CHECKLIST                       │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  1. PARSING — Is your text extraction accurate?                  │
│     □ Tables render correctly                                    │
│     □ No garbled text from bad PDF extraction                    │
│     □ Metadata (titles, dates) preserved                         │
│                                                                  │
│  2. CHUNKING — Are chunks meaningful units?                      │
│     □ Chunks don't split mid-sentence or mid-paragraph           │
│     □ Chunk size matches your embedding model's sweet spot       │
│     □ Overlap prevents losing context at boundaries              │
│     □ Semantic chunking used for unstructured content            │
│                                                                  │
│  3. EMBEDDING — Is your model right for the data?                │
│     □ Same model for indexing AND querying                       │
│     □ Instruction prefixes used correctly (if applicable)        │
│     □ Tested on domain-specific queries (not just MTEB)          │
```

```
│                                                                  │
│  4. RETRIEVAL — Are you finding the right chunks?                │
│     □ Hybrid search (vector + BM25) enabled                      │
│     □ Reranking applied to top-N candidates                      │
│     □ Metadata filters used to narrow scope                      │
│     □ Retrieve 3-5 chunks (not too many, not too few)            │
│                                                                  │
│  5. GENERATION — Is the LLM using context well?                  │
│     □ System prompt instructs to use provided context            │
│     □ "If not in context, say you don't know"                    │
│     □ Context fits within token budget (< 50% of window)         │
│     □ Source attribution in responses                            │
│                                                                  │
│  6. EVALUATION — Are you measuring quality?                      │
│     □ Retrieval: Recall@K, MRR, NDCG                             │
│     □ Generation: Faithfulness, relevance, groundedness          │
│     □ Use RAGAS or LangSmith for automated eval                  │
│     □ Human evaluation for critical use cases                    │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

A.6 Common RAG Failure Modes & Fixes

Symptom	Likely Cause	Fix
"I don't know" when answer IS in the data	Chunks don't match query semantically	Try hybrid search, rephrase queries, or add reranking
Hallucinated answer despite good retrieval	LLM ignores context or context is noisy	Stronger system prompt: "ONLY use provided context"
Answer is vague or generic	Chunks are too large or too many	Reduce chunk size, use semantic chunking
Missing context from middle of a document	Chunk split at bad boundary	Add overlap, use document-structure-aware splitter
Slow retrieval	No ANN index, or brute-force search	Build HNSW/IVF index, tune ef_search/nprobe
Good for short queries, bad for long queries	Embedding model has short max tokens	Use long-context model (BGE-M3, Jina v3: 8K)
Works in English, fails in other languages	Embedding model is English-only	Use multilingual model (BGE-M3, Cohere embed-v3)
Numbers/dates are wrong in responses	Semantic search is bad with numbers	Add structured metadata filters (date >, price <)

A.7 Production Architecture Decision Tree

```
Building a RAG system? Follow this decision tree:

  ┌─ What stage are you in?
  │
  ├─ Prototype / MVP
  │   → ChromaDB + LangChain + OpenAI embeddings
  │   → Simplest setup, get results in hours
  │
  ├─ Production (< 10M vectors)
  │   │
  │   ├─ On Azure?
  │   │   → Azure AI Search + Azure OpenAI + Semantic Kernel
  │   │   → Or: Cosmos DB (if vectors live with app data)
  │   │
  │   ├─ Already using PostgreSQL?
  │   │   → pgvector + your existing stack
  │   │
  │   ├─ Want managed (no ops)?
  │   │   → Pinecone or Weaviate Cloud
  │   │
  │   └─ Want self-hosted + fast?
  │       → Qdrant (single node or small cluster)
  │
  ├─ Production (10M - 1B vectors)
  │   │
  │   ├─ On Azure?
  │   │   → Azure AI Search (scales to billions with partitions)
  │   │
  │   ├─ Want managed?
  │   │   → Pinecone (pods) or Zilliz Cloud (Milvus)
  │   │
  │   └─ Self-hosted?
  │       → Milvus cluster or Qdrant cluster
  │
  └─ Billion+ scale
      → Milvus cluster + IVF-PQ + GPU indexes
      → Or: custom FAISS pipeline with DiskANN

  ┌─ On Azure? The full Azure RAG stack:
  │
  │   Parsing:       Azure Document Intelligence (Layout model)
  │   Embedding:     Azure OpenAI text-embedding-3-*
  │   Vector Store:  Azure AI Search (hybrid + semantic ranker)
  │   Orchestration: Semantic Kernel (.NET) or LangChain (Python)
  │   LLM:           Azure OpenAI GPT-4o / GPT-4.1
  │   Auth:          Entra ID + Managed Identity (no API keys)
  │   Zero-code:     Azure OpenAI "On Your Data" feature
  │

  ┌─ Orchestration framework?
  │
  ├─ .NET / Azure-first → Semantic Kernel (Microsoft-backed)
  ├─ Quick prototype    → LangChain (widest integrations)
  ├─ Complex data/queries → LlamaIndex (better retrieval strategies)
```

```
├─ Custom pipeline      → Build your own (embed → store → retrieve)
└─ Agent workflows      → LangGraph (stateful, multi-step)
```